In [1]:
```
import pandas as pd
import numpy as np
import matplotlib .pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
df=pd.read_excel("insurance.xlsx")
```

In [3]:
```
df                              # to show the top 5 and last 5 rows
```

Out[3]:

	months_as_customer	age	policy_number	policy_bind_date	policy_state	policy_csl	policy_deductable	policy_annual_premium	umbrella_limit	insur
0	328	48	521585	2014-10-17	OH	250/500	1000	1406.91	0	
1	228	42	342868	2006-06-27	IN	250/500	2000	1197.22	5000000	
2	134	29	687698	2000-09-06	OH	100/300	2000	1413.14	5000000	
3	256	41	227811	1990-05-25	IL	250/500	2000	1415.74	6000000	
4	228	44	367455	2014-06-06	IL	500/1000	1000	1583.91	6000000	
...	...	...	...	...	...	...	...	...	...	
995	3	38	941851	1991-07-16	OH	500/1000	1000	1310.80	0	
996	285	41	186934	2014-01-05	IL	100/300	1000	1436.79	0	
997	130	34	918516	2003-02-17	OH	250/500	500	1383.49	3000000	
998	458	62	533940	2011-11-18	IL	500/1000	2000	1356.92	5000000	
999	456	60	556080	1996-11-11	OH	250/500	1000	766.19	0	

1000 rows × 39 columns

In [4]:
```
# Title :-       "Insurance Claim Fraud Detection"
```

In [5]:
```
# Domain:-      "Finance"
```

In [6]:
```
# Description:- "This project explores insurance claim data to find signs of fraud.
#                It uses simple analysis and charts to understand what makes a claim suspicious."
```

In [7]:
```
# Skills:-      "1 Python","2 Pandas","3 Statistics","4 Data manipulation","5 Data Analysis",
#               "6 Data transformation","7 Data cleaning","8 Data visualization","9 Project methodology"
```

Business Problem understanding

In [9]:
```
#  To pridicting Insurance Claim Fraud Detection
```

Data understanding and exploration

In [11]:
```
df.shape              #  to check the total rows and columns
```

Out[11]:
```
(1000, 39)
```

In [12]:
```
# Observe : there are 1000 rows and 39 columns
```

In [13]:
```
df.dtypes            # to check the data types of each columns
```

Out[13]:
```
months_as_customer                int64
age                               int64
policy_number                     int64
policy_bind_date         datetime64[ns]
policy_state                     object
policy_csl                       object
policy_deductable                 int64
policy_annual_premium           float64
umbrella_limit                    int64
insured_zip                       int64
insured_sex                      object
insured_education_level          object
insured_occupation               object
insured_hobbies                  object
insured_relationship             object
capital-gains                     int64
capital-loss                      int64
incident_date            datetime64[ns]
incident_type                    object
collision_type                   object
incident_severity                object
authorities_contacted            object
incident_state                   object
incident_city                    object
incident_location                object
incident_hour_of_the_day          int64
number_of_vehicles_involved       int64
property_damage                  object
bodily_injuries                   int64
witnesses                         int64
police_report_available          object
total_claim_amount                int64
injury_claim                      int64
property_claim                    int64
vehicle_claim                     int64
auto_make                        object
auto_model                       object
auto_year                         int64
fraud_reported                   object
dtype: object
```

In [14]:
```
# observe :  there are  17 int, 19 object,1 float,2 datatime datatypes columns
```

In [15]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 39 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   months_as_customer           1000 non-null   int64
 1   age                          1000 non-null   int64
 2   policy_number                1000 non-null   int64
 3   policy_bind_date             1000 non-null   datetime64[ns]
 4   policy_state                 1000 non-null   object
 5   policy_csl                   1000 non-null   object
 6   policy_deductable            1000 non-null   int64
 7   policy_annual_premium        1000 non-null   float64
 8   umbrella_limit               1000 non-null   int64
 9   insured_zip                  1000 non-null   int64
 10  insured_sex                  1000 non-null   object
 11  insured_education_level      1000 non-null   object
 12  insured_occupation           1000 non-null   object
 13  insured_hobbies              1000 non-null   object
 14  insured_relationship         1000 non-null   object
 15  capital-gains                1000 non-null   int64
 16  capital-loss                 1000 non-null   int64
 17  incident_date                1000 non-null   datetime64[ns]
 18  incident_type                1000 non-null   object
 19  collision_type               1000 non-null   object
 20  incident_severity            1000 non-null   object
 21  authorities_contacted        909 non-null    object
 22  incident_state               1000 non-null   object
 23  incident_city                1000 non-null   object
 24  incident_location            1000 non-null   object
 25  incident_hour_of_the_day     1000 non-null   int64
 26  number_of_vehicles_involved  1000 non-null   int64
 27  property_damage              1000 non-null   object
 28  bodily_injuries              1000 non-null   int64
 29  witnesses                    1000 non-null   int64
 30  police_report_available      1000 non-null   object
 31  total_claim_amount           1000 non-null   int64
 32  injury_claim                 1000 non-null   int64
 33  property_claim               1000 non-null   int64
 34  vehicle_claim                1000 non-null   int64
 35  auto_make                    1000 non-null   object
 36  auto_model                   1000 non-null   object
 37  auto_year                    1000 non-null   int64
 38  fraud_reported               1000 non-null   object
dtypes: datetime64[ns](2), float64(1), int64(17), object(19)
memory usage: 304.8+ KB
```

In [16]:
```
# Obesreve: to show the rows ,columns,null value and data type of each columns
```

In [17]:
```
df.isnull().sum()           # to check the null values of each columns
```

Out[17]:
```
months_as_customer            0
age                           0
policy_number                 0
policy_bind_date              0
policy_state                  0
policy_csl                    0
policy_deductable             0
policy_annual_premium         0
umbrella_limit                0
insured_zip                   0
insured_sex                   0
insured_education_level       0
insured_occupation            0
insured_hobbies               0
insured_relationship          0
capital-gains                 0
capital-loss                  0
incident_date                 0
incident_type                 0
collision_type                0
incident_severity             0
authorities_contacted        91
incident_state                0
incident_city                 0
incident_location             0
incident_hour_of_the_day      0
number_of_vehicles_involved   0
property_damage               0
bodily_injuries               0
witnesses                     0
police_report_available       0
total_claim_amount            0
injury_claim                  0
property_claim                0
vehicle_claim                 0
auto_make                     0
auto_model                    0
auto_year                     0
fraud_reported                0
dtype: int64
```

In [18]:
```
# observe : only one columns have null value "authorities_contacted column"
```

In [19]:
```
df.columns.tolist()
```

Out[19]:
```
['months_as_customer',
 'age',
 'policy_number',
 'policy_bind_date',
 'policy_state',
 'policy_csl',
 'policy_deductable',
 'policy_annual_premium',
 'umbrella_limit',
 'insured_zip',
 'insured_sex',
 'insured_education_level',
 'insured_occupation',
 'insured_hobbies',
 'insured_relationship',
 'capital-gains',
 'capital-loss',
 'incident_date',
 'incident_type',
 'collision_type',
 'incident_severity',
 'authorities_contacted',
 'incident_state',
 'incident_city',
 'incident_location',
 'incident_hour_of_the_day',
 'number_of_vehicles_involved',
 'property_damage',
 'bodily_injuries',
 'witnesses',
 'police_report_available',
 'total_claim_amount',
 'injury_claim',
 'property_claim',
 'vehicle_claim',
 'auto_make',
 'auto_model',
 'auto_year',
 'fraud_reported']
```

In [20]:
```
# Obesreve: To show all columns name in a list
# explain each columns
# Customer Information

# months_as_customer: Number of months the individual has been a customer.
#age: Age of the insured customer.
#policy_number: Unique identifier for the insurance policy.
#policy_bind_date: The date when the insurance policy became effective.
#policy_state: The U.S. state in which the policy was issued.
#policy_csl: Combined Single Limit; a policy limit for bodily injury and property damage per accident.
#policy_deductable: The amount the insured must pay out of pocket before the insurer covers the rest.
#policy_annual_premium: The annual premium (cost) for the policy.
#umbrella_limit: Additional liability coverage that goes beyond the limits of other policies.

#Insured Person Information

#insured_zip: Zip code of the insured person.
#insured_sex: Gender of the insured (e.g., male, female).
#insured_education_level: Education level of the insured person.
#insured_occupation: Job title or occupation of the insured.
#insured_hobbies: Hobbies of the insured, which may correlate with risk.
#insured_relationship: Relationship status (e.g., single, married).

#Financial Info

#capital-gains: Profit earned from the sale of assets or investments.
#capital-loss: Loss incurred from the sale of assets or investments.

#Incident Details

#incident_date: Date when the incident (e.g., accident) occurred.
#incident_type: Type of incident (e.g., single vehicle collision, multi-vehicle collision).
#collision_type: Specifics of the collision (e.g., rear-end, side swipe).
#incident_severity: Severity of the incident (e.g., minor damage, total loss).
#authorities_contacted: Whether police or emergency authorities were contacted.
#incident_state: State where the incident occurred.
#incident_city: City where the incident occurred.
#incident_location: Exact location or address of the incident.
#incident_hour_of_the_day: Hour during the day (0-23) the incident happened.
#number_of_vehicles_involved: Total number of vehicles involved in the incident.
#property_damage: Whether property damage occurred (Yes/No).
#bodily_injuries: Number of bodily injuries reported.
#witnesses: Number of witnesses present at the incident.
#police_report_available: Whether a police report was filed (Yes/No).

#Claim Information
```

```
#total_claim_amount: Total monetary value of the claim.
#injury_claim: Amount claimed for injuries.
#property_claim: Amount claimed for property damage.
#vehicle_claim: Amount claimed for vehicle repair/replacement.

#Vehicle Information

#auto_make: Brand of the insured vehicle (e.g., Toyota, Ford).
#auto_model: Model of the vehicle (e.g., Camry, F-150).
#auto_year: Manufacturing year of the vehicle.

#Target Variable

#fraud_reported: Whether the claim was reported as fraud (Yes/No).
```

In [21]:
```
df.duplicated().sum()
```

Out[21]:
```
0
```

In [22]:
```
# Obesreve: To check all rows for any duplicates values
```

In [23]:
```
df.head()
```

Out[23]:

	months_as_customer	age	policy_number	policy_bind_date	policy_state	policy_csl	policy_deductable	policy_annual_premium	umbrella_limit	insured
0	328	48	521585	2014-10-17	OH	250/500	1000	1406.91	0	466
1	228	42	342868	2006-06-27	IN	250/500	2000	1197.22	5000000	468
2	134	29	687698	2000-09-06	OH	100/300	2000	1413.14	5000000	430
3	256	41	227811	1990-05-25	IL	250/500	2000	1415.74	6000000	608
4	228	44	367455	2014-06-06	IL	500/1000	1000	1583.91	6000000	610

5 rows × 39 columns

In [24]:
```
# Obeserve: top 5 rows by default selects
```

In [25]:
```
df.tail()
```

Out[25]:

	months_as_customer	age	policy_number	policy_bind_date	policy_state	policy_csl	policy_deductable	policy_annual_premium	umbrella_limit	insur
995	3	38	941851	1991-07-16	OH	500/1000	1000	1310.80	0	
996	285	41	186934	2014-01-05	IL	100/300	1000	1436.79	0	
997	130	34	918516	2003-02-17	OH	250/500	500	1383.49	3000000	
998	458	62	533940	2011-11-18	IL	500/1000	2000	1356.92	5000000	
999	456	60	556080	1996-11-11	OH	250/500	1000	766.19	0	

5 rows × 39 columns

In [26]:
```
# Obeserve:  5 buttom rows by default selects
```

Explore each column wise

column 1= months_as_customer (that means Number of months the person has been a customer.)

In [29]:
```
df["months_as_customer"].dtypes
```

Out[29]:
```
dtype('int64')
```

In [30]:
```
# Observe : this column is int Datatype,count/ discrete variable  and correct Datatypes
```

In [31]:
```
df["months_as_customer"].isnull().sum()
```

Out[31]:
```
0
```

In [32]:
```
# Observe : in this column there is no null value
```

In [33]:
```
df["months_as_customer"].unique()
```

Out[33]:
```
array([328, 228, 134, 256, 137, 165,  27, 212, 235, 447,  60, 121, 180,
       473,  70, 140, 160, 196, 460, 217, 370, 413, 237,   8, 257, 202,
       224, 241,  64, 166, 155, 114, 149, 147,  62, 289, 431, 199,  79,
       116,  37, 106, 269, 265, 163, 355, 175, 192, 430,  91, 223, 195,
        22, 439,  94,  11, 151, 154, 245, 119, 215, 295, 254, 107, 478,
       128, 338, 271, 222, 120, 270, 319, 194, 227, 244,  78, 200, 284,
       275, 153,  31,  41, 127,  61, 207, 219,  80, 325,  29, 239, 279,
       350, 464, 118, 298,  87, 261, 453, 210, 168, 390, 258, 225, 164,
       255, 206, 203, 211, 274,  81, 280, 112,  24,  93, 171, 124, 287,
       122, 398, 214, 209,  82, 193, 288, 104, 101, 375, 461, 428,  45,
       136, 216, 278, 108,  14, 276,  47,  73, 294, 324,  53, 426, 111,
        86, 296, 125, 177, 238, 449, 252, 359,  19, 285,  30, 342, 468,
       343, 404,  63, 335, 142, 272,  69,  38, 281, 246, 330, 362, 371,
       377, 172,  99, 249, 190, 174,  95,   2, 117, 242, 440,  20, 208,
       156, 232,  84, 394,  35, 369, 332, 243, 264,  32, 259, 186, 201,
       436, 189, 105,  88,  40,  59,  39, 123, 231, 247,  48, 267, 286,
       253,  10, 158,   1,  85, 233, 266,  97, 399, 305, 129, 283,  96,
       176, 159, 290, 299,  66, 334, 429,  15, 230, 250,  65, 475,  77,
       229, 110, 292, 451, 150, 291, 162, 309, 396, 273, 419, 315,  72,
       157, 113, 115, 236,   7, 126, 297, 406, 146, 409,   6, 103, 344,
       437,  56, 277, 138,   4, 167, 467, 310, 143, 102, 303, 152, 144,
       414, 352,  33, 347, 410, 448, 218,  43, 411, 360, 300,  67, 380,
       389, 317, 191, 321,   0, 405, 304,  26,  12,  83, 184, 479,   5,
       220, 316, 379, 354, 169, 339, 131, 313, 337, 204, 364, 185, 251,
       392, 402, 182, 282, 415,  51, 130, 463, 472,  75,   3,  98,  16,
       438, 422,  34, 183, 356, 109, 198, 446, 435, 434,  25, 148, 145,
       263,  46, 234, 456,  58, 293, 385, 100, 373, 322,  76,  54,  92,
       173, 366, 188,  89, 458, 161, 476, 133, 320, 421, 135,  57, 187,
       386, 197, 179, 132,   9, 427,  13, 312, 260, 141, 441, 381, 178,
       425,  55,  90,  36, 308, 412, 465, 407, 205, 372, 213, 240,  50,
        17], dtype=int64)
```

In [34]:
```
df["months_as_customer"].nunique()
```

Out[34]:
```
391
```

In [35]:
```
# check the column which has different  unique value
```

In [36]:
```
df["months_as_customer"].value_counts()
```

Out[36]:
```
months_as_customer
194    8
128    7
254    7
140    7
210    7
      ..
390    1
411    1
453    1
448    1
17     1
Name: count, Length: 391, dtype: int64
```

In [37]:
```
#among these 194 months  has more customer.
```

column 2 = "age"

In [39]:
```
df["age"].unique()
```

Out[39]:
```
array([48, 42, 29, 41, 44, 39, 34, 37, 33, 61, 23, 38, 58, 26, 31, 62, 55,
       40, 35, 43, 45, 25, 30, 28, 49, 54, 47, 59, 27, 56, 32, 36, 64, 60,
       51, 46, 50, 57, 53, 24, 52, 19, 21, 63, 20, 22], dtype=int64)
```

In [40]:
```
# Observation: age is a continuous features Wide range of values
```

In [41]:
```
df["age"].isnull().sum()
```

Out[41]:
```
0
```

In [42]:
```
# Observe : in this column there is no null value
```

column 3= "policy_number" (that means Unique ID of the insurance policy)

In [44]:
```
df["policy_number"].dtypes
```

Out[44]:
```
dtype('int64')
```

In [45]:
```
# Observe : this column is int Datatype,continuous variable and correct Datatypes
```

In [46]:
```
df["policy_number"].unique()
```

Out[46]:
```
array([521585, 342868, 687698, 227811, 367455, 104594, 413978, 429027,
       485665, 636550, 543610, 214618, 842643, 626808, 644081, 892874,
       558938, 275265, 921202, 143972, 183430, 431876, 285496, 115399,
       736882, 699044, 863236, 608513, 914088, 596785, 908616, 666333,
       336614, 584859, 990493, 129872, 200152, 933293, 485664, 982871,
       206213, 616337, 448961, 790442, 108844, 430029, 529112, 939631,
       866931, 582011, 691189, 537546, 394975, 729634, 282195, 420810,
       524836, 307195, 623648, 485372, 598554, 303987, 343161, 519312,
       132902, 332867, 356590, 346002, 500533, 348209, 486676, 260845,
       657045, 761189, 175177, 116700, 166264, 527945, 627540, 279422,
       484200, 645258, 694662, 960680, 498140, 498875, 798177, 614763,
       679370, 958857, 686816, 127754, 918629, 731450, 307447, 992145,
       900628, 235220, 740019, 246882, 797613, 193442, 389238, 760179,
       939905, 872814, 632627, 283414, 163161, 853360, 776860, 149367,
       395269, 981123, 143626, 648397, 154982, 330591, 319232, 531640,
       368050, 253791, 155724, 824540, 717392, 965768, 414779, 428230,
       517240, 469874, 718428, 620215, 618659, 649082, 437573, 964657,
       932502, 434507, 935277, 756054, 682387, 456604, 139872, 354105,
       165485, 515050, 795686, 395983, 119513, 217938, 203914, 565157,
       904191, 419510, 575000, 120485, 781181, 299796, 589749, 854021,
       454086, 139484, 678849, 346940, 985436, 237418, 335780, 491392,
       140880, 962591, 922565, 288580, 154280, 425973, 477177, 648509,
       914815, 249048, 144323, 651861, 125324, 398102, 514065, 391652,
       922167, 442795, 226330, 134430, 524230, 438817, 293794, 868283,
       828890, 882920, 918777, 212580, 602410, 976971, 630226, 171254,
       247116, 505969, 653864, 586367, 896890, 650026, 547744, 598124,
       436126, 739447, 427484, 218684, 565564, 743163, 604614, 509928,
       593390, 970607, 174701, 529398, 940942, 442677, 365364, 114839,
       872734, 267885, 740505, 629663, 839884, 241562, 405533, 667021,
       511621, 476923, 735822, 492745, 130930, 261119, 280709, 898573,
       547802, 600845, 390381, 629918, 208298, 513099, 184938, 187775,
       326322, 146138, 336047, 532330, 118137, 212674, 935596, 737593,
       812025, 168151, 594739, 843227, 283925, 475588, 751905, 226725,
       942504, 395572, 889883, 818167, 277767, 842618, 577810, 873114,
       994538, 727792, 522506, 367595, 586104, 424862, 512813, 356768,
       330506, 779075, 799501, 987905, 967756, 830414, 127313, 786957,
       332892, 448642, 526039, 444422, 689500, 806081, 384618, 756459,
       655787, 419954, 275092, 515698, 132871, 714929, 297816, 426708,
       615047, 771236, 235869, 931625, 371635, 427199, 261315, 582973,
       278091, 153154, 515217, 860497, 351741, 403737, 162004, 740384,
       876714, 951543, 576723, 391003, 225865, 984948, 890328, 803294,
       414913, 414519, 818413, 487356, 159768, 865839, 406567, 623032,
       935442, 106873, 563878, 620855, 583169, 337677, 445973, 156694,
       421940, 613226, 804410, 553565, 399524, 331595, 380067, 701521,
       360770, 958785, 797934, 883980, 340614, 435784, 563837, 200827,
       533941, 265026, 354481, 566720, 832746, 386690, 979285, 594722,
```

216738, 369048, 514424, 954191, 150181, 388671, 457244, 206667,
745200, 412703, 736771, 347984, 626074, 218109, 999435, 858060,
500384, 903785, 873859, 204294, 467106, 357713, 890026, 751612,
876680, 756981, 121439, 411289, 538466, 932097, 463727, 552618,
936638, 348814, 944102, 689901, 901083, 396224, 682178, 596298,
253005, 985924, 631565, 630998, 926665, 302669, 620020, 439828,
971295, 165565, 936543, 296960, 501692, 525224, 355085, 830729,
651948, 424358, 131478, 268833, 287489, 808153, 687639, 497347,
439660, 847123, 172307, 810189, 432068, 903203, 253085, 180720,
492224, 411477, 107181, 312940, 855186, 373935, 812989, 993840,
327856, 506333, 263159, 372912, 552788, 722747, 248467, 955953,
910622, 137675, 343421, 413192, 247801, 171147, 431283, 461962,
149467, 758740, 628337, 574637, 373600, 930032, 396590, 238412,
484321, 795847, 218456, 792673, 662256, 971338, 714738, 753844,
976645, 918037, 996253, 373731, 836272, 167231, 743330, 807369,
735307, 789208, 585324, 498759, 795004, 203250, 430794, 156636,
284143, 740518, 445289, 599262, 357949, 493161, 320251, 231127,
766193, 555374, 491484, 925128, 265093, 267808, 116735, 963680,
445694, 215534, 232854, 168260, 538955, 243226, 246435, 582480,
345539, 924318, 726880, 190588, 246705, 619589, 164988, 729534,
505014, 920826, 534982, 110408, 283052, 840806, 382394, 876699,
871432, 379882, 852002, 372891, 689034, 743092, 599174, 421092,
349658, 811042, 505316, 116645, 950880, 788502, 627486, 498842,
550294, 328387, 540152, 385932, 618682, 550930, 998192, 663938,
756870, 337158, 919875, 315631, 113464, 556415, 250249, 403776,
396002, 976908, 509489, 485295, 361829, 603632, 783494, 439049,
502634, 378588, 794731, 641934, 113516, 425631, 542245, 512894,
633090, 464234, 290162, 638155, 911429, 106186, 311783, 528385,
228403, 209177, 497929, 735844, 710741, 276804, 507545, 485642,
796375, 171183, 110084, 714784, 143924, 996850, 284834, 830878,
270208, 407958, 832300, 927205, 655356, 831053, 432740, 893853,
594988, 979544, 191891, 831479, 714346, 326289, 944537, 779156,
856153, 473338, 521694, 136520, 730819, 912665, 469966, 952300,
322609, 890280, 431583, 155912, 110143, 808544, 409074, 824728,
606037, 636843, 111874, 439844, 463513, 577858, 607351, 682754,
757352, 307469, 526296, 658816, 913337, 488464, 480094, 263108,
298412, 261905, 674485, 223404, 991480, 804219, 483088, 100804,
941807, 593466, 437442, 942106, 794951, 182450, 730973, 687755,
757644, 998865, 944953, 386429, 108270, 205134, 749325, 774303,
698470, 719989, 309323, 444035, 431478, 797634, 284836, 238196,
885789, 287436, 496067, 206004, 153027, 469426, 654974, 943425,
641845, 794534, 357808, 536052, 873384, 790225, 587498, 639027,
217899, 589094, 458829, 626208, 315041, 283267, 442494, 159243,
669800, 520179, 607974, 465065, 369941, 447226, 831668, 922937,
640474, 153298, 334749, 221283, 961496, 804751, 369226, 691115,
713172, 621756, 615116, 947598, 658002, 374545, 805806, 235097,

```
 290971, 180286, 662088, 884365, 178081, 507452, 990624, 892148,
 398683, 605100, 143109, 230223, 769602, 420815, 973546, 608039,
 250162, 786432, 445195, 938634, 482495, 796005, 910604, 327488,
 715202, 648852, 516959, 984456, 801331, 786103, 684193, 247505,
 259792, 185124, 760700, 362407, 389525, 179538, 265437, 266247,
 921851, 488724, 192524, 338070, 865607, 963285, 728491, 553436,
 440616, 463237, 753452, 920554, 594783, 725330, 607259, 979336,
 865201, 140977, 787351, 272330, 728025, 804608, 718829, 482404,
 331170, 753056, 910365, 379268, 362843, 135400, 798579, 250833,
 824116, 322613, 871305, 488037, 485813, 886473, 907113, 833321,
 521592, 254837, 634499, 574707, 476839, 149601, 630683, 500639,
 352120, 569245, 907012, 700074, 866805, 951863, 211578, 357394,
 863749, 596914, 684653, 528259, 797636, 326180, 620075, 965187,
 516182, 728839, 771509, 264221, 602704, 672416, 545506, 777533,
 953334, 369781, 990998, 982678, 646069, 331683, 364055, 521854,
 737252, 344480, 898519, 957816, 175960, 489618, 717044, 101421,
 793948, 737483, 695117, 167466, 664732, 143038, 979963, 467841,
 219028, 130156, 762951, 376879, 599031, 676255, 985446, 884180,
 571462, 815883, 258265, 569714, 180008, 633375, 556538, 669501,
 963761, 753005, 454758, 698589, 330119, 164464, 927354, 231508,
 272910, 305758, 950542, 291544, 388616, 577992, 342830, 491170,
 175553, 439341, 221186, 868031, 844117, 744557, 159536, 727109,
 155604, 608443, 117862, 991553, 727443, 378587, 420948, 457188,
 118236, 987524, 490596, 524215, 913464, 398484, 752504, 449263,
 844007, 686522, 670142, 607687, 967713, 291902, 149839, 840225,
 643226, 535879, 746630, 598308, 720356, 724752, 148498, 110122,
 281388, 728600, 231548, 531160, 889003, 193213, 557218, 125591,
 227244, 791425, 354455, 601042, 433663, 471938, 564654, 645723,
 573572, 437960, 649800, 544225, 390256, 488597, 133889, 931901,
 769475, 844062, 844129, 732169, 221854, 776950, 291006, 845751,
 889764, 929306, 515457, 556270, 908935, 525862, 490514, 774895,
 974522, 669809, 182953, 836349, 591269, 550127, 663190, 109392,
 215278, 674570, 681486, 941851, 186934, 918516, 533940, 556080],
dtype=int64)
```

In [47]:
```
df["policy_number"].nunique()
```

Out[47]:
```
1000
```

In [48]:
```
#Not useful for analysis  but  it helps in tracking records. so that
# simply drop them
```

column 4 ="policy_bind_date" (that means Date when the policy was created or started.)

In [50]:
```
df["policy_bind_date"].dtypes
```

Out[50]:
```
dtype('<M8[ns]')
```

In [51]:
```
# Observe : this column is datetime Datatype and correct Datatypes
```

In [52]:
```
df["policy_bind_date"].nunique()
```

Out[52]:
```
951
```

In [53]:
```
# almost each rows has unique value (if more then 30% of the rows values uni.) there is no use in the analysis)
# simply drop them
```

column 5 ="policy_state"

In [55]:
```
df["policy_state"].dtypes
```

Out[55]:
```
dtype('O')
```

In [56]:
```
# Observe : this column is discrete variable,object  Datatype and correct Datatypes
```

In [57]:
```
df["policy_state"].value_counts()
```

Out[57]:
```
policy_state
OH    352
IL    338
IN    310
Name: count, dtype: int64
```

In [58]:
```
# Observe : this columan has three types of value OH contains more
```

In [59]:
```
df["policy_state"].isnull().sum()
```

Out[59]:
```
0
```

In [60]:
```
# Observe : in this column there is no null value
```

In [61]:
```
df["policy_state"].unique()
```

Out[61]:
```
array(['OH', 'IN', 'IL'], dtype=object)
```

In [62]:
```
df["policy_state"].nunique()
```

Out[62]:
```
3
```

In [63]:
```
# Observe : this column has three types of value  contains
```

column 6= "policy_csl" that means Combined single limit for bodily injury/property damage

In [65]:
```
df["policy_csl"].unique()
```

Out[65]:
```
array(['250/500', '100/300', '500/1000'], dtype=object)
```

In [66]:
```
# policy_csl is categorical features indicating the policy_csl of the patients
       #with three categories :'250/500', '100/300', '500/1000'
```

In [67]:
```
df["policy_csl"].value_counts()
```

Out[67]:
```
policy_csl
250/500     351
100/300     349
500/1000    300
Name: count, dtype: int64
```

In [68]:
```
# Observe : this column has three types of value 250/500 contains more
```

In [69]:
```
df["policy_csl"].isnull().sum()
```

Out[69]:
```
0
```

In [70]:
```
# Observe : in this column there is no null value
```

column 7="policy_deductable" (that means The deductible amount to be paid by the policyholder.)

In [72]:
```
df["policy_deductable"].dtypes
```

Out[72]:
```
dtype('int64')
```

In [73]:
```
# Observe : this column is int Datatype,count/discrete variable and correct Datatypes
```

In [74]:
```
df["policy_deductable"].isnull().sum()
```

Out[74]:
```
0
```

In [75]:
```
# Observe : in this column there is no null value
```

In [76]:
```
df["policy_deductable"].unique()
```

Out[76]:
```
array([1000, 2000,  500], dtype=int64)
```

In [77]:
```
# Observe : this column has three types of value  contains
```

In [78]:
```
df["policy_deductable"].value_counts()
```

Out[78]:
```
policy_deductable
1000    351
500     342
2000    307
Name: count, dtype: int64
```

In [79]:
```
# Observe : this column has three types of value 1000 contains more
```

column 8= "policy_annual_premium" (Yearly premium paid by the insured person)

In [81]:
```
df["policy_annual_premium"].dtypes
```

Out[81]:
```
dtype('float64')
```

In [82]:
```
# Observe : this column is float Datatype,continuous variable and correct Datatypes
```

In [83]:
```
df["policy_annual_premium"].isnull().sum()
```

Out[83]:
```
0
```

In [84]:
```
# Observe : in this column there is no null value
```

In [85]:
```
df["policy_annual_premium"].unique()
```

Out[85]:
```
array([1406.91, 1197.22, 1413.14, 1415.74, 1583.91, 1351.1 , 1333.35,
       1137.03, 1442.99, 1315.68, 1253.12, 1137.16, 1215.36,  936.61,
       1301.13, 1131.4 , 1199.44,  708.64, 1374.22, 1475.73, 1187.96,
        875.15,  972.18, 1268.79,  883.31, 1266.92, 1322.1 ,  848.07,
       1291.7 , 1104.5 ,  954.16, 1337.28, 1088.34, 1558.29, 1415.68,
       1334.15,  988.45, 1222.48, 1155.55, 1262.08, 1451.62, 1737.66,
       1475.93,  538.17, 1081.08, 1454.43, 1240.47, 1273.7 , 1123.87,
       1245.89, 1326.62, 1073.83, 1530.52, 1201.41, 1393.57, 1276.57,
       1082.49, 1414.74, 1470.06,  870.63,  795.23, 1168.2 ,  993.51,
       1848.81, 1641.73, 1362.87, 1239.22,  835.02, 1061.33, 1279.08,
       1105.49, 1055.53,  895.83, 1632.93, 1405.99, 1425.54, 1038.09,
       1307.11, 1489.24,  976.67, 1340.43, 1267.81, 1234.2 , 1318.06,
        769.95, 1514.72,  873.64, 1612.43, 1318.24, 1226.83, 1326.44,
       1136.83, 1322.78, 1483.25, 1515.3 , 1075.18, 1690.27, 1352.83,
       1148.73,  969.5 , 1463.82, 1474.17, 1497.35, 1427.14, 1495.1 ,
       1141.62, 1125.37, 1207.36, 1338.5 , 1074.07, 1337.56, 1298.91,
       1222.75, 1059.52, 1124.38, 1110.37, 1103.58, 1269.76,  964.79,
       1167.3 , 1625.45, 1394.43, 1053.24, 1040.75, 1302.4 , 1588.55,
       1399.26, 1352.31, 1139.  , 1397.67,  823.17,  965.13, 1922.84,
       1624.82, 1277.25, 1439.34, 1281.27, 1348.83, 1198.15, 1221.22,
        968.74, 1220.71, 1238.62, 1320.75,  990.98, 1398.51, 1355.08,
       1384.51,  847.03, 1000.06, 1046.71, 1158.03, 1372.27, 1053.04,
       1275.39, 1402.75, 1344.36, 1197.71, 1203.24, 1152.4 , 1142.62,
       1332.07, 1166.54, 1495.06, 1337.92, 1587.96, 1362.29, 1448.84,
       1241.97, 1124.6 , 1079.92, 1447.78, 1229.16, 1226.49,  897.89,
       1706.79, 1254.18,  885.08, 1046.58, 1712.68, 1097.71, 1363.77,
       1382.88, 1141.35, 1054.83, 1057.77, 1488.02,  920.3 ,  986.53,
       1440.68, 1086.21, 1367.68, 1215.85, 1191.19, 1594.45, 1463.07,
       1734.09, 1411.43, 1512.58, 1153.35, 1722.95, 1281.07, 1011.92,
       1042.26, 1235.1 ,  768.91, 1301.72, 1451.54,  767.14, 1620.89,
       1048.46, 1538.26, 1217.69, 1362.64, 1279.13,  924.72, 1019.44,
       1314.6 , 1515.18, 1649.18, 1451.01,  978.46, 1198.34, 1003.23,
       1212.  , 1242.96, 1053.02, 1693.63, 2047.59, 1083.72, 1138.42,
       1072.62, 1219.04, 1371.78, 1506.21, 1058.21,  932.14, 1608.34,
       1728.56, 1518.46, 1540.19,  965.21, 1278.75,  773.99, 1532.47,
       1340.56, 1297.75,  433.33, 1025.54, 1264.77, 1459.97, 1238.65,
       1050.76, 1711.72,  865.33, 1153.49, 1281.25, 1342.8 , 1443.32,
       1629.94, 1134.08, 1483.91, 1304.67, 1035.79, 1401.2 , 1665.45,
        653.66, 1080.13, 1346.18, 1589.54, 1251.65, 1083.01,  974.59,
       1399.85, 1307.74, 1219.27, 1411.3 ,  694.45, 1006.77, 1422.36,
       1348.32, 1315.56, 1407.01, 1388.58, 1310.76, 1004.63, 1134.91,
       1194.  , 1248.25, 1338.54,  782.23, 1366.9 , 1275.81, 1090.65,
       1326.  ,  972.47,  806.31, 1416.24, 1097.64,  947.75, 1018.73,
       1400.74, 1155.53, 1386.93,  915.29, 1239.55, 1366.42, 1086.48,
       1247.87, 1312.75,  765.64, 1327.41, 1338.55, 1316.63, 1286.44,
       1372.18, 1447.77,  903.32, 1454.42, 1603.42, 1616.58, 1611.83,
```

 889.13, 1252.08, 995.56, 1347.92, 1724.09, 1379.93, 1554.64,
1377.94, 1313.33, 1259.02, 1371.88, 1399.27, 1061.98, 1365.46,
 894.4 , 956.69, 1123.89, 1085.03, 1437.33, 988.29, 1238.89,
1384.64, 1595.07, 1127.89, 929.7 , 1829.63, 904.7 , 1243.84,
1030.95, 1285.03, 1216.56, 966.26, 1203.17, 1212.12, 1573.93,
1609.67, 1097.57, 1618.65, 922.67, 1342.02, 1195.01, 994.74,
1050.24, 1313.51, 1102.29, 1185.78, 1527.95, 1366.39, 1403.9 ,
 927.23, 1554.86, 736.07, 974.16, 973.8 , 1260.32, 1464.03,
 617.11, 1724.46, 1161.31, 1091.73, 1209.07, 1241.04, 1757.21,
 802.24, 1342.72, 1209.41, 1141.71, 1090.03, 1464.73, 1118.58,
1117.04, 1257.83, 1082.72, 1191.8 , 1242.02, 1075.71, 969.88,
1459.93, 1245.61, 1462.76, 1398.46, 1269.64, 1455.65, 1140.31,
1330.46, 1190.6 , 972.5 , 1161.91, 1117.17, 1101.51, 1523.17,
 984.45, 1257. , 1434.51, 1628. , 1412.31, 1134.68, 1306.78,
1021.9 , 1538.6 , 1354.5 , 1282.93, 1346.27, 1338.4 , 949.44,
 977.4 , 1181.46, 1187.53, 845.16, 1442.27, 1276.43, 1075.41,
1111.72, 814.96, 1575.86, 1115.27, 1175.7 , 793.15, 942.51,
1056.71, 1255.68, 1335.13, 1178.95, 1793.16, 1008.38, 1396.83,
1402.78, 1437.88, 1313.64, 1482.14, 1171.75, 1353.33, 1175.51,
1836.02, 1480.79, 1453.92, 1340.71, 714.03, 1376.16, 914.22,
1601.47, 1096.79, 1078.22, 1595.28, 1217.84, 1117.42, 1567.37,
1294.93, 1152.12, 1441.21, 1575.74, 1233.96, 1861.43, 1570.86,
 791.47, 1012.78, 1047.06, 1390.29, 951.46, 1226.78, 1280.9 ,
1472.77, 1878.44, 1418.5 , 1532.8 , 1304.35, 1551.61, 1326.98,
 862.92, 1331.69, 1486.04, 870.55, 1344.56, 1377.04, 1237.88,
1525.86, 854.58, 770.76, 1132.74, 1173.37, 1188.28, 876.88,
1143.95, 1409.06, 1070.63, 916.13, 1191.5 , 1474.66, 1193.45,
1437.53, 1304.83, 1168.8 , 1164.97, 1232.72, 1800.76, 1187.01,
1559.34, 1137.02, 1281.72, 796.35, 1270.02, 1383.13, 1290.74,
1216.68, 1251.16, 1586.41, 1526.11, 1028.44, 1555.94, 1570.77,
1170.53, 1099.95, 1472.43, 1275.62, 1292.3 , 1009.37, 1093.07,
1325.44, 1017.18, 1221.17, 1927.87, 978.27, 1221.14, 1255.62,
 999.52, 1380.89, 1010.77, 1205.86, 1526.61, 1496.44, 1256.2 ,
1268.35, 1421.59, 1500.04, 1433.24, 1231.25, 1135.43, 945.73,
1118.76, 1231.98, 1005.47, 1108.97, 1392.39, 1317.97, 1588.22,
 900.02, 1744.64, 1260.56, 1021.14, 1285.09, 1537.07, 1022.42,
1558.86, 1757.87, 973.5 , 1430.8 , 1192.27, 1163.83, 1003.15,
1153.54, 1631.1 , 1252.48, 1677.26, 979.73, 989.24, 1389.86,
1233.85, 1320.39, 1435.09, 1448.54, 1733.56, 1533.07, 1106.77,
 995.7 , 1298.85, 1276.73, 1202.28, 671.92, 1358.03, 1008.79,
1141.1 , 1397. , 1664.66, 1151.39, 1546.01, 1063.67, 709.14,
1039.55, 1339.39, 1202.75, 1081.17, 991.39, 984.02, 1354.83,
 830.31, 987.42, 1119.29, 1048.39, 1074.47, 1230.76, 1255.02,
1555.52, 836.11, 1450.98, 625.08, 1133.27, 1366.6 , 1439.9 ,
1230.69, 1307.68, 1124.69, 1520.78, 1609.11, 1358.91, 1295.87,
1161.03, 1441.06, 1097.99, 1464.42, 1543.68, 1390.89, 1148.58,

1616.26, 1398.94, 1238.92, 968.46, 1045.12, 1537.33, 912.3 ,
1007.28, 1189.98, 1576.41, 1172.82, 1312.22, 671.01, 895.14,
1373.21, 1625.65, 1295.63, 1459.96, 1219.94, 1064.49, 959.83,
1767.02, 1285.01, 1422.95, 1223.39, 1539.06, 988.06, 1740.57,
1540.91, 1381.88, 1446.98, 1220.86, 948.1 , 1471.24, 1157.97,
 566.11, 1060.88, 1524.45, 1556.17, 1216.24, 1484.72, 1203.81,
1393.34, 1484.48, 1581.27, 1667.83, 1497.41, 803.36, 1221.41,
1865.83, 1434.27, 1114.68, 1218.56, 1234.69, 964.92, 1351.72,
 817.28, 1389.59, 1353.53, 1294.04, 840.81, 998.19, 1090.32,
1780.67, 1681.01, 1395.77, 1471.44, 1265.72, 1274.7 , 1330.39,
1122.95, 1655.79, 935.77, 1192.04, 990.11, 1422.21, 914.85,
1123.84, 838.02, 1300.68, 1173.21, 985.97, 1129.23, 1194.83,
1114.29, 1509.04, 664.86, 1267.4 , 1119.23, 1698.51, 1422.78,
1212.75, 1423.34, 976.37, 1124.59, 1569.33, 1359.36, 1607.36,
1042.25, 1453.95, 1969.63, 1156.19, 1124.47, 1493.5 , 1155.38,
 878.19, 1145.85, 1261.28, 1427.46, 1592.41, 1274.63, 1362.31,
 919.37, 1377.23, 995.55, 1508.12, 484.67, 1561.41, 1457.21,
1128.71, 1358.2 , 1232.79, 936.19, 1302.34, 1264.99, 1467.76,
1124.43, 1319.81, 1482.53, 1328.18, 1463.95, 1133.85, 1037.32,
1562.8 , 1425.79, 1615.14, 1236.5 , 1017.97, 1306. , 1174.14,
1231.01, 1299.18, 1469.75, 1390.72, 1694.09, 1140.15, 1310.71,
1730.49, 1616.65, 1935.85, 855.14, 1568.47, 1550.53, 1370.92,
1363.59, 828.42, 1209.63, 1111.17, 989.97, 1114.23, 1347.31,
1687.53, 1183.48, 1537.13, 1173.25, 1361.16, 1422.56, 1143.06,
1405.71, 1354.2 , 1055.6 , 999.43, 1155.97, 1726.91, 1232.57,
1078.65, 1324.78, 1518.54, 1239.06, 1246.68, 1622.67, 1082.36,
1270.55, 1236.32, 785.82, 1265.84, 1508.9 , 1106.84, 1389.13,
 974.84, 1304.46, 1257.36, 1257.04, 719.52, 1524.18, 1395.58,
1243.68, 1189.04, 1375.29, 1387.51, 1178.61, 1166.62, 1556.31,
1452.27, 1212.07, 1578.54, 1488.26, 1096.39, 1482.57, 954.18,
1193.4 , 1023.11, 1653.32, 1022.46, 1396.43, 1521.55, 1034.27,
1255.35, 1396.89, 795.31, 982.22, 1311.3 , 1277.12, 1388.62,
1406.52, 1132.47, 1896.91, 1143.46, 1285.42, 1305.26, 1051.67,
1387.35, 1083.64, 1851.78, 1270.29, 1459.99, 1253.44, 1142.48,
1188.45, 1125.4 , 1294.41, 1459.5 , 1367.99, 1594.37, 1035.99,
 911.53, 1319.97, 1391.63, 915.41, 1468.82, 1412.76, 1588.26,
1140.91, 1517.54, 912.29, 1144.3 , 1281.43, 1101.85, 1082.1 ,
1185.44, 1175.07, 979.26, 920.81, 922.85, 1107.59, 1272.46,
1340.24, 1648. , 1381.14, 1198.44, 951.27, 1341.24, 1177.57,
1055.09, 1479.48, 1827.38, 1169.62, 1516.34, 1270.21, 809.11,
1115.81, 1457.65, 1041.36, 1693.83, 1685.69, 1054.92, 1333.97,
1013.61, 958.3 , 1112.04, 1206.26, 763.67, 1042.56, 1263.48,
1006.99, 1328.26, 1250.08, 982.7 , 1412.06, 1066.7 , 1585.54,
1416.08, 1246.03, 1356.64, 1387.7 , 1004.14, 1107.07, 1429.96,
1074.99, 1007. , 1052.85, 1200.33, 1343. , 1441.6 , 1433.42,
1368.57, 862.19, 871.46, 1863.04, 1181.64, 1060.74, 951.56,

```
1533.71, 1200.09, 1093.83,  988.93, 1331.94, 1361.45, 1188.51,
1101.83,  840.95, 1503.21, 1722.5 ,  944.03, 1453.61, 1672.88,
1248.05, 1564.43, 1280.88,  722.66, 1235.14, 1347.04, 1310.8 ,
1436.79, 1383.49, 1356.92,  766.19])
```

In [86]:
```
df["policy_annual_premium"].nunique()
```

Out[86]:
```
991
```

In [87]:
```
# observe : - almost each rows have unique value but this is impact the analysis  so not drop them
```

column 9= "umbrella_limit" (Extra liability coverage beyond regular policy limits)

In [89]:
```
df["umbrella_limit"].dtypes
```

Out[89]:
```
dtype('int64')
```

In [90]:
```
# Observe : this column is int Datatype,continuous variable and correct Datatypes
```

In [91]:
```
df["umbrella_limit"].isnull().sum()
```

Out[91]:
```
0
```

In [92]:
```
# Observe : in this column there is no null value
```

In [93]:
```
df["umbrella_limit"].unique()
```

Out[93]:
```
array([       0,  5000000,  6000000,  4000000,  3000000,  8000000,
        7000000,  9000000, 10000000, -1000000,  2000000], dtype=int64)
```

In [94]:
```
df["umbrella_limit"].nunique()
```

Out[94]:
```
11
```

In [95]:
```
df["umbrella_limit"].value_counts()
```

Out[95]:
```
umbrella_limit
 0           798
 6000000      57
 5000000      46
 4000000      39
 7000000      29
 3000000      12
 8000000       8
 9000000       5
 2000000       3
 10000000      2
-1000000       1
Name: count, dtype: int64
```

In [96]:
```
# observe : - there is one value in incorrect format
```

column 10= "insured_zip" (ZIP code of the insured person (very high cardinality))

In [98]:
```
df["insured_zip"].dtypes
```

Out[98]:
```
dtype('int64')
```

In [99]:
```
# Observe : this column is int Datatype,continuous variable and correct Datatypes
```

In [100…
```
df["insured_zip"].unique()
```

Out[100…

```
array([466132, 468176, 430632, 608117, 610706, 478456, 441716, 603195,
       601734, 600983, 462283, 615561, 432220, 464652, 476685, 458733,
       619884, 470610, 472135, 477670, 618845, 442479, 443920, 453148,
       434733, 613982, 436984, 607730, 609837, 432211, 473328, 610393,
       614780, 472248, 603381, 479224, 430141, 620757, 615901, 474615,
       456446, 470577, 441648, 433782, 468104, 459407, 472573, 433473,
       446326, 435481, 477310, 609930, 603993, 437818, 478423, 467784,
       606714, 464691, 431683, 431725, 609216, 452787, 468767, 435489,
       450149, 458364, 476458, 602433, 478575, 449718, 463181, 441992,
       452597, 614417, 472895, 475847, 476978, 600648, 608335, 471600,
       441175, 603123, 457767, 618498, 605486, 617970, 432934, 456762,
       601748, 607763, 436973, 471300, 453277, 465100, 603248, 601112,
       438830, 464959, 439787, 464839, 448984, 440327, 460742, 446895,
       609374, 451672, 604450, 432896, 618929, 451312, 605141, 459504,
       432781, 452748, 618316, 455365, 470603, 475292, 467743, 460675,
       618123, 607452, 606352, 603527, 445601, 603948, 435758, 611586,
       465263, 617858, 607889, 455689, 450341, 431277, 454656, 605169,
       444822, 447442, 474360, 447925, 451586, 477519, 603639, 463993,
       441491, 469429, 472214, 614945, 476727, 438555, 440961, 616714,
       434247, 436547, 619540, 466283, 449557, 473329, 470117, 600702,
       615921, 475588, 609409, 479852, 452249, 441536, 601617, 442598,
       430987, 430104, 612904, 430886, 467947, 604804, 460308, 618862,
       462479, 457555, 459984, 434982, 614233, 605258, 604377, 434923,
       476456, 446788, 477382, 600275, 461958, 472720, 442395, 455340,
       613247, 454985, 468813, 452747, 615611, 451400, 464874, 452496,
       430714, 472634, 608371, 468168, 464107, 466959, 443522, 441726,
       473412, 466201, 469621, 466676, 615346, 440106, 450332, 615226,
       437688, 437387, 458139, 443191, 613647, 460820, 431121, 619735,
       470485, 620473, 449800, 602402, 452456, 439269, 617774, 477678,
       444913, 456602, 451560, 453407, 618655, 612550, 466718, 617947,
       606238, 463842, 610354, 461328, 458727, 452587, 433184, 451280,
       603269, 442632, 447300, 441783, 468702, 467942, 463678, 615220,
       432711, 463583, 439502, 613287, 620104, 431531, 605408, 457551,
       619892, 445853, 475483, 606290, 611852, 444734, 433683, 448882,
       466838, 605490, 466137, 466970, 474801, 450703, 478172, 604668,
       471806, 475705, 459122, 476737, 460359, 452735, 613583, 605692,
       438178, 449221, 459322, 472657, 608331, 438546, 441981, 602177,
       441659, 614812, 458470, 469646, 611118, 465158, 457130, 607893,
       464736, 476198, 444903, 464336, 471453, 466191, 440930, 430380,
       613178, 460564, 439929, 605756, 451184, 459588, 616637, 447979,
       460176, 459429, 465456, 464665, 430853, 615712, 608228, 457535,
       442540, 455332, 439534, 462420, 448913, 440837, 466634, 446435,
       438237, 468313, 476303, 450339, 476502, 600561, 600754, 439304,
       460722, 618632, 452204, 454530, 474848, 435985, 457942, 436522,
       471704, 455810, 446544, 461919, 470128, 462836, 475407, 473611,
       608425, 476227, 452701, 456789, 600904, 450889, 478837, 611322,
```

438180, 449793, 450730, 608758, 445339, 438328, 479913, 460760,
444797, 436711, 432491, 617527, 601213, 604138, 431361, 477695,
612597, 445638, 476185, 435995, 430232, 443861, 460801, 605121,
458622, 478661, 435299, 601961, 604328, 614385, 438584, 478703,
615683, 455672, 602942, 616706, 473243, 435552, 434206, 469895,
457722, 473645, 619108, 610479, 474998, 616341, 460535, 606487,
620737, 445904, 464145, 466818, 464237, 618455, 616126, 468508,
431937, 448603, 444500, 601117, 615383, 434342, 435100, 431278,
445648, 448857, 435267, 461275, 613816, 608767, 620869, 478981,
464630, 466303, 452647, 441370, 619166, 472803, 442308, 469383,
614383, 438617, 613936, 472163, 447458, 474792, 470559, 432399,
607605, 600153, 465979, 466555, 444155, 465764, 446898, 453274,
479320, 443462, 446158, 602514, 477356, 434669, 609322, 614265,
606177, 461514, 454685, 477260, 469126, 443402, 479408, 467227,
468433, 604289, 471366, 450746, 614948, 473935, 617267, 470670,
450368, 448809, 469653, 615688, 465631, 443344, 441363, 462683,
463184, 612826, 433155, 616120, 461744, 475916, 454434, 464353,
610302, 462106, 431389, 442866, 446755, 464743, 437889, 473638,
444232, 458237, 441499, 613436, 448912, 468872, 619811, 614166,
456600, 618405, 430832, 610989, 447750, 608708, 469650, 602304,
459878, 441142, 465667, 473109, 619794, 602258, 479724, 442210,
463291, 474898, 431354, 617460, 609317, 479821, 473394, 603882,
615229, 620197, 438215, 444583, 471866, 616884, 448310, 478902,
442695, 613826, 476203, 604333, 442604, 435663, 470866, 612908,
441871, 431496, 436499, 469853, 605369, 448466, 432786, 473591,
618418, 444558, 457733, 466161, 450800, 458993, 468634, 461264,
600184, 604874, 462377, 619657, 437323, 432148, 439690, 601848,
615821, 472475, 457463, 604861, 471519, 618682, 441425, 609336,
603320, 615446, 435967, 610246, 479327, 468300, 612660, 466691,
468515, 614521, 465921, 604555, 616276, 463356, 450184, 466393,
471786, 602289, 445120, 449260, 472724, 475173, 443854, 461418,
616164, 620962, 465201, 470488, 462250, 436408, 464230, 478609,
437156, 432218, 620493, 475391, 440720, 606942, 446971, 470538,
601177, 451470, 438529, 469742, 435534, 442239, 468986, 606988,
453719, 475524, 617804, 613399, 453400, 615767, 615311, 468470,
461383, 457727, 613327, 614941, 440680, 609949, 479655, 439964,
478486, 466498, 430878, 600127, 431968, 462804, 435809, 453193,
459630, 608982, 452218, 434150, 460579, 442142, 608807, 433153,
436560, 436784, 430621, 601574, 433853, 453164, 613931, 607458,
463835, 613945, 432699, 613119, 472922, 613849, 603827, 467780,
460586, 613842, 435371, 466289, 436173, 457234, 474758, 477373,
613471, 601581, 612102, 460263, 479134, 451467, 602670, 613607,
611556, 435518, 465942, 446174, 611651, 446657, 612506, 618493,
612664, 473653, 454529, 437422, 619470, 442666, 620507, 614867,
609898, 450702, 600418, 431202, 457793, 470190, 603733, 465136,
611723, 608963, 454139, 447560, 444378, 616583, 455913, 454399,

```
602842, 459428, 613114, 450709, 444626, 601206, 470389, 615218,
606249, 616161, 442335, 604952, 441533, 471784, 453265, 444922,
474324, 441298, 446606, 459537, 440757, 604948, 433275, 608309,
462767, 471785, 601397, 477636, 441967, 454776, 431532, 614169,
601425, 477346, 613587, 620358, 617699, 430567, 439870, 438837,
458997, 604147, 606638, 619620, 441671, 610381, 602416, 459562,
463271, 458132, 448949, 603732, 608929, 469875, 443342, 456363,
470826, 458582, 454480, 435632, 442206, 468303, 467762, 447188,
469438, 462519, 432534, 436467, 465674, 442389, 471614, 442936,
437944, 473705, 469363, 465376, 438775, 457962, 477947, 431104,
456570, 612986, 615730, 478640, 470510, 439360, 440251, 600313,
452216, 478532, 616929, 620207, 605743, 472814, 464362, 456203,
468984, 473349, 474771, 448294, 606606, 605220, 466612, 463331,
457843, 609226, 452942, 609390, 446608, 602500, 463809, 611996,
459298, 468158, 440831, 603848, 617739, 607133, 437470, 461372,
476130, 452438, 460517, 444896, 448722, 477856, 617721, 454176,
618127, 441923, 604279, 440833, 451550, 431853, 614274, 619148,
456781, 434293, 460895, 601600, 465440, 455482, 438877, 479824,
477415, 614372, 465248, 449421, 445856, 608525, 608813, 459295,
606144, 476315, 475891, 462525, 606283, 465252, 449979, 604681,
466390, 612316, 474731, 603260, 434370, 478388, 617883, 464808,
609458, 432405, 457875, 477268, 437580, 457121, 436364, 467654,
471148, 468202, 456959, 447274, 608405, 472253, 438923, 607131,
601701, 469220, 433250, 444413, 433593, 458143, 474167, 476413,
600208, 618926, 606219, 448436, 447976, 472236, 468232, 620819,
452349, 464646, 472209, 459955, 473389, 616767, 442948, 458936,
613921, 474598, 440865, 450947, 473370, 463153, 612546, 442919,
449352, 470104, 459889, 478868, 463307, 453620, 466238, 607697,
477631, 443625, 472223, 608328, 474860, 606858, 477938, 462698,
454552, 471585, 455426, 469856, 454191, 468454, 614187, 433974,
604833, 447469, 451529, 448190, 453713, 440153, 619570, 478947,
443550, 477644, 433981, 433696, 443567, 430665, 431289, 608177,
442797, 441714, 612260], dtype=int64)
```

In [101…

```
# Nearly unique; adds little value unless regional analysis so that simply drop them
```

In [102…

```
pd.set_option('display.max_columns', None)
#  observe : - show all rows and columns
```

columns 11="insured_sex" (Gender of the insured person (MALE/FEMALE))

In [104…

```
df["insured_sex"].dtypes
```

Out[104… dtype('O')

In [105…
```
# Observe : this column is object  Datatype discrete variable  and correct Datatypes
```

In [106…
```
df["insured_sex"].value_counts()
```

Out[106…
```
insured_sex
FEMALE    537
MALE      463
Name: count, dtype: int64
```

In [107…
```
# Observe : this column has FEMALE value  contains MORE
```

In [108…
```
df["insured_sex"].isnull().sum()
```

Out[108… 0

In [109…
```
# Observe : in this column there is no null value
```

In [110…
```
df["insured_sex"].unique()
```

Out[110… array(['MALE', 'FEMALE'], dtype=object)

In [111…
```
# this columns contains 2 type
```

column 12 ="insured_education_level" (Highest education level of the insured)

In [113…
```
df["insured_education_level"].dtypes
```

Out[113… dtype('O')

In [114…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [115…
```
df["insured_education_level"].value_counts()
```

Out[115…
```
insured_education_level
JD             161
High School    160
Associate      145
MD             144
Masters        143
PhD            125
College        122
Name: count, dtype: int64
```

In [116…
```
# Observe : this column have JD  value  contains MORE
```

In [117…
```
df["insured_education_level"].isnull().sum()
```

Out[117…
```
0
```

In [118…
```
# Observe : in this column there is no null value
```

In [119…
```
df["insured_education_level"].nunique()
```

Out[119…
```
7
```

In [120…
```
# this columns contains 7 type
```

column 13="insured_occupation" (Job type of the insured)

In [122…
```
df["insured_occupation"].dtypes
```

Out[122…
```
dtype('O')
```

In [123…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [124…
```
df["insured_occupation"].value_counts()
```

Out[124…
```
insured_occupation
machine-op-inspct    93
prof-specialty       85
tech-support         78
sales                76
exec-managerial      76
craft-repair         74
transport-moving     72
other-service        71
priv-house-serv      71
armed-forces         69
adm-clerical         65
protective-serv      63
handlers-cleaners    54
farming-fishing      53
Name: count, dtype: int64
```

In [125…
```
# Observe : this column have machine-op-inspct  value  contains MORE
```

In [126…
```
df["insured_occupation"].isnull().sum()
```

Out[126…
```
0
```

In [127…
```
# Observe : in this column there is no null value
```

In [128…
```
df["insured_occupation"].nunique()
```

Out[128…
```
14
```

In [129…
```
# this columns contains 14 type
```

column 14 = "insured_hobbies" (Listed hobbies of the insured person)

In [131…
```
df["insured_hobbies"].dtypes
```

Out[131…
```
dtype('O')
```

In [132…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [133…
```
df["insured_hobbies"].value_counts()
```

Out[133…
```
insured_hobbies
reading           64
exercise          57
paintball         57
bungie-jumping    56
movies            55
golf              55
camping           55
kayaking          54
yachting          53
hiking            52
video-games       50
skydiving         49
base-jumping      49
board-games       48
polo              47
chess             46
dancing           43
sleeping          41
cross-fit         35
basketball        34
Name: count, dtype: int64
```

In [134…
```
# Observe : this column have reading value  contains MORE
```

In [135…
```
df["insured_hobbies"].isnull().sum()
```

Out[135…
```
0
```

In [136…
```
# Observe : in this column there is no null value
```

In [137…
```
df["insured_hobbies"].nunique()
```

Out[137…
```
20
```

In [138…
```
# this columns contains 20 type
```

column 15= "insured_relationship" (Relationship of the insured person in the household)

In [140…
```
df["insured_relationship"].dtypes
```

Out[140… dtype('O')

In [141…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [142…
```
df["insured_relationship"].value_counts()
```

Out[142…
```
insured_relationship
own-child          183
other-relative     177
not-in-family      174
husband            170
wife               155
unmarried          141
Name: count, dtype: int64
```

In [143…
```
# Observe : this column have own-child value  contains MORE
```

In [144…
```
df["insured_relationship"].isnull().sum()
```

Out[144… 0

In [145…
```
# Observe : in this column there is no null value
```

In [146…
```
df["insured_relationship"].nunique()
```

Out[146… 6

In [147…
```
# this columns contains 6 type
```

column 16= "capital-gains" (Income gained from capital assets)

In [149…
```
df["capital-gains"].dtypes
```

Out[149… dtype('int64')

In [150…
```
# Observe : this column is int Datatype,continuous variable and correct Datatypes
```

In [151…
```
df["capital-gains"].isnull().sum()
```

Out[151… 0

In [152…
```
# Observe : in this column there is no null value
```

column 17="capital-loss"

In [154…
```
df["capital-loss"].dtypes
```

Out[154…
```
dtype('int64')
```

In [155…
```
# Observe : this column is int Datatype,continuous variable and correct Datatypes
```

In [156…
```
df["capital-loss"].isnull().sum()
```

Out[156…
```
0
```

In [157…
```
# Observe : in this column there is no null value
```

column 18 = "incident_date"

In [159…
```
df["incident_date"].dtypes
```

Out[159…
```
dtype('<M8[ns]')
```

In [160…
```
# Observe : this column is int Date time ,time series variable and correct Datatypes
```

In [161…
```
df["incident_date"].isnull().sum()
```

Out[161…
```
0
```

In [162…
```
# Observe : in this column there is no null value
```

column 19="incident_type"

In [164…
```
df["incident_type"].dtypes
```

Out[164…
```
dtype('O')
```

In [165…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [166…
```
df["incident_type"].isnull().sum()
```

Out[166…
```
0
```

In [167…
```
# Observe : in this column there is no null value
```

In [168…
```
df["incident_type"].value_counts()
```

Out[168…
```
incident_type
Multi-vehicle Collision     419
Single Vehicle Collision    403
Vehicle Theft                94
Parked Car                   84
Name: count, dtype: int64
```

In [169…
```
# Observe : this column have Multi-vehicle Collision value  contains MORE
```

In [170…
```
df["incident_type"].nunique()
```

Out[170…
```
4
```

In [171…
```
# this columns contains 6 type
```

column 20="collision_type"

In [173…
```
df["collision_type"].dtypes
```

Out[173…
```
dtype('O')
```

In [174…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [175…
```
df["collision_type"].isnull().sum()
```

Out[175…
```
0
```

In [176…
```
# Observe : in this column there is no null value
```

In [177…
```
df["collision_type"].value_counts()
```

Out[177…
```
collision_type
Rear Collision     292
Side Collision     276
Front Collision    254
?                  178
Name: count, dtype: int64
```

In [178…
```
# Observe : this column have rear Collision value  contains MORE
```

In [179…
```
df["collision_type"].nunique()
```

Out[179…
```
4
```

In [180…
```
# this columns contains 4 type
```

column 21 ="incident_severity"

In [182…
```
df["incident_severity"].dtypes
```

Out[182…
```
dtype('O')
```

In [183…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [184…
```
df["incident_severity"].isnull().sum()
```

Out[184…
```
0
```

In [185…
```
# Observe : in this column there is no null value
```

In [186…
```
df["incident_severity"].value_counts()
```

Out[186…
```
incident_severity
Minor Damage      354
Total Loss        280
Major Damage      276
Trivial Damage     90
Name: count, dtype: int64
```

In [187…
```
# Observe : this column have Minor Damage  contains MORE value
```

In [188…
```
df["incident_severity"].nunique()
```

Out[188… 4

In [189… # this columns contains 4 type

columns 22 = "authorities_contacted"

In [191… df["authorities_contacted"].dtypes

Out[191… dtype('O')

In [192… # Observe : this column is object Datatype discrete variable and correct Datatypes

In [193… df["authorities_contacted"].isnull().sum()

Out[193… 91

In [194… # Observe : in this column there is 91 null value

In [195… df["authorities_contacted"].value_counts()

Out[195…
```
authorities_contacted
Police       292
Fire         223
Other        198
Ambulance    196
Name: count, dtype: int64
```

In [196… # Observe : this column have police value contains MORE

In [197… df["authorities_contacted"].unique()

Out[197… array(['Police', nan, 'Fire', 'Other', 'Ambulance'], dtype=object)

In [198… # this columns contains 4 type and some nan

column 23 ="incident_state"

In [200… df["incident_state"].dtypes

Out[200… dtype('O')

In [201…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [202…
```
df["incident_state"].isnull().sum()
```

Out[202… 0

In [203…
```
# Observe : in this column there is no null value
```

In [204…
```
df["incident_state"].value_counts()
```

Out[204…
```
incident_state
NY    262
SC    248
WV    217
VA    110
NC    110
PA     30
OH     23
Name: count, dtype: int64
```

In [205…
```
# Observe : this column have NY value  contains MORE
```

In [206…
```
df["incident_state"].nunique()
```

Out[206… 7

In [207…
```
# this columns contains 7 type
```

column 24="incident_city"

In [209…
```
df["incident_city"].dtypes
```

Out[209… dtype('O')

In [210…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [211…
```
df["incident_city"].isnull().sum()
```

Out[211… 0

In [212…
```
# Observe : in this column there is no null value
```

In [213…
```
df["incident_city"].value_counts()
```

Out[213…
```
incident_city
Springfield    157
Arlington      152
Columbus       149
Northbend      145
Hillsdale      141
Riverwood      134
Northbrook     122
Name: count, dtype: int64
```

In [214…
```
# Observe : this column have Springfield  value  contains MORE
```

In [215…
```
df["incident_city"].nunique()
```

Out[215…
```
7
```

In [216…
```
# this columns contains 7 type
```

columns 25=" incident_location"

In [218…
```
df["incident_location"].dtypes
```

Out[218…
```
dtype('O')
```

In [219…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [220…
```
df["incident_location"].nunique()
```

Out[220…
```
1000
```

In [221…
```
#each rows has unique value (if more then 30% of the rows values uni.) there is no use in the analysis)
# simply drop them
```

column 26="incident_hour_of_the_day"

In [223…
```
df["incident_hour_of_the_day"].dtypes
```

Out[223…
```
dtype('int64')
```

In [224…
```
# Observe : this column is int  Datatype count/discrete variable  and correct Datatypes
```

In [225…
```
df["incident_hour_of_the_day"].value_counts()
```

Out[225…
```
incident_hour_of_the_day
17    54
3     53
0     52
23    51
16    49
13    46
10    46
4     46
6     44
9     43
14    43
21    42
18    41
12    40
19    40
7     40
15    39
22    38
8     36
20    34
5     33
2     31
11    30
1     29
Name: count, dtype: int64
```

In [226…
```
# Observe : this column have Springfield  value  contains MORE
```

In [227…
```
df["incident_hour_of_the_day"].isnull().sum()
```

Out[227…
```
0
```

In [228…
```
# Observe : in this column there is no null value
```

In [229…
```
df["incident_hour_of_the_day"].nunique()
```

Out[229… 24

In [230…
```
# this columns contains 24 type
```

columns 27="number_of_vehicles_involved"

In [232…
```
df["number_of_vehicles_involved"].dtypes
```

Out[232… dtype('int64')

In [233…
```
# Observe : this column is int  Datatype count/discrete variable  and correct Datatypes
```

In [234…
```
df["number_of_vehicles_involved"].isnull().sum()
```

Out[234… 0

In [235…
```
# Observe : in this column there is no null value
```

In [236…
```
df["number_of_vehicles_involved"].value_counts()
```

Out[236…
```
number_of_vehicles_involved
1    581
3    358
4     31
2     30
Name: count, dtype: int64
```

In [237…
```
# Observe : this column have 1 value  contains MORE
```

In [238…
```
df["number_of_vehicles_involved"].nunique()
```

Out[238… 4

In [239…
```
# this columns contains 4 type
```

columns 28="property_damage"

In [241…
```
df["property_damage"].dtypes
```

Out[241…
```
dtype('O')
```

In [242…
```
# Observe : this column is object Datatype discrete variable  and correct Datatypes
```

In [243…
```
df["property_damage"].value_counts()
```

Out[243…
```
property_damage
?      360
NO     338
YES    302
Name: count, dtype: int64
```

In [244…
```
# Observe : this column have ? value  contains MORE
```

In [245…
```
df["property_damage"].isnull().sum()
```

Out[245…
```
0
```

In [246…
```
# Observe : in this column there is no null value
```

In [247…
```
df["property_damage"].unique()
```

Out[247…
```
array(['YES', '?', 'NO'], dtype=object)
```

In [248…
```
# this columns contains 2  type and one is ?
```

columns 29= "bodily_injuries"

In [250…
```
df["bodily_injuries"].dtypes
```

Out[250…
```
dtype('int64')
```

In [251…
```
# Observe : this column is int Datatype,count/ discrete variable  and correct Datatypes
```

In [252…
```
df["bodily_injuries"].isnull().sum()
```

Out[252…
```
0
```

In [253…
```
# Observe : in this column there is no null value
```

In [254…
```
df["bodily_injuries"].value_counts()
```

Out[254…
```
bodily_injuries
0    340
2    332
1    328
Name: count, dtype: int64
```

In [255…
```
# Observe : this column has 0 value  contains more
```

In [256…
```
df["bodily_injuries"].unique()
```

Out[256…
```
array([1, 0, 2], dtype=int64)
```

In [257…
```
# Observe : this column has 3 types of value  contains
```

column 30= "witnesses" (Number of witnesses present)

In [259…
```
df["witnesses"].dtypes
```

Out[259…
```
dtype('int64')
```

In [260…
```
# Observe : this column is int Datatype,count/ discrete variable  and correct Datatypes
```

In [261…
```
df["witnesses"].unique()
```

Out[261…
```
array([2, 0, 3, 1], dtype=int64)
```

In [262…
```
# Observe : this column has four types of value  contains
```

In [263…
```
df["witnesses"].isnull().sum()
```

Out[263…
```
0
```

In [264…
```
# Observe : in this column there is no null value
```

column 31= "police_report_available"

In [266…
```
df["police_report_available"].dtypes
```

Out[266… dtype('O')

In [267… # Observe : this column is discrete variable,object Datatype and correct Datatypes

In [268… df["police_report_available"].value_counts()

Out[268…
```
police_report_available
?      343
NO     343
YES    314
Name: count, dtype: int64
```

In [269… # Observe : this column has three types of value contains

In [270… df["police_report_available"].isnull().sum()

Out[270… 0

In [271… # Observe : in this column there is no null value

column 32= "total_claim_amount" (Total amount claimed from the insurance)

In [273… df["total_claim_amount"].dtypes

Out[273… dtype('int64')

In [274… # Observe : this column is int Datatype,continuous variable and correct Datatypes

In [275… df["total_claim_amount"].isnull().sum()

Out[275… 0

In [276… # Observe : in this column there is no null value

column 33 ="injury_claim" (Amount claimed for injuries)

In [278… df["injury_claim"].dtypes

Out[278… dtype('int64')

In [279…
```
# Observe : this column is int Datatype,continuous variable and correct Datatypes
```

In [280…
```
df["injury_claim"].isnull().sum()
```

Out[280… 0

In [281…
```
# Observe : in this column there is no null value
```

In [282…
```
df["injury_claim"].nunique()
```

Out[282… 638

In [283…
```
# observe : - almost each rows have unique value but this is impact the analysis  so not drop them
```

column 34 ="property_claim" (Claim amount for property damage.)

In [285…
```
df["property_claim"].dtypes
```

Out[285… dtype('int64')

In [286…
```
# Observe : this column is int Datatype,continuous variable and correct Datatypes
```

In [287…
```
df["property_claim"].isnull().sum()
```

Out[287… 0

In [288…
```
# Observe : in this column there is no null value
```

column 35= "vehicle_claim" (Claim amount for vehicle repair or replacement.)

In [290…
```
df["vehicle_claim"].dtypes
```

Out[290… dtype('int64')

In [291…
```
# Observe : this column is int Datatype,continuous variable and correct Datatypes
```

In [292…
```
df["vehicle_claim"].isnull().sum()
```

Out[292…
```
0
```

In [293…
```
# Observe : in this column there is no null value
```

column 36 = "auto_make" (Car manufacturer)

In [295…
```
df["auto_make"].dtypes
```

Out[295…
```
dtype('O')
```

In [296…
```
# Observe : this column is object Datatype,discrete variable and correct Datatypes
```

In [297…
```
df["auto_make"].isnull().sum()
```

Out[297…
```
0
```

In [298…
```
# Observe : in this column there is no null value
```

In [299…
```
df["auto_make"].value_counts()
```

Out[299…
```
auto_make
Saab          80
Dodge         80
Suburu        80
Nissan        78
Chevrolet     76
Ford          72
BMW           72
Toyota        70
Audi          69
Accura        68
Volkswagen    68
Jeep          67
Mercedes      65
Honda         55
Name: count, dtype: int64
```

In [300…
```
# Observe : this column has 14 types of value  contains
```

column 37= "auto_model" (Model of the car.)

In [302…
```
df["auto_model"].dtypes
```

Out[302…
```
dtype('O')
```

In [303…
```
# Observe : this column is object Datatype,discrete variable and correct Datatypes
```

In [304…
```
df["auto_model"].isnull().sum()
```

Out[304…
```
0
```

In [305…
```
# Observe : in this column there is no null value
```

In [306…
```
df["auto_model"].value_counts()
```

Out[306…

```
auto_model
RAM               43
Wrangler          42
A3                37
Neon              37
MDX               36
Jetta             35
Passat            33
A5                32
Legacy            32
Pathfinder        31
Malibu            30
92x               28
Camry             28
Forrestor         28
F150              27
95                27
E400              27
93                25
Grand Cherokee    25
Escape            24
Tahoe             24
Maxima            24
Ultima            23
X5                23
Highlander        22
Civic             22
Silverado         22
Fusion            21
ML350             20
Impreza           20
Corolla           20
TL                20
CRV               20
C300              18
3 Series          18
X6                16
M5                15
Accord            13
RSX               12
Name: count, dtype: int64
```

In [307…

```
# Observe : this column has 14 types of value  contains
```

column 38= "auto_year" (Year of the car model)

In [309…
```
df["auto_year"].dtypes
```

Out[309…
```
dtype('int64')
```

In [310…
```
# Observe : this column is object Datatype,count/discrete variable and correct Datatypes
```

In [311…
```
df["auto_year"].value_counts()
```

Out[311…
```
auto_year
1995    56
1999    55
2005    54
2006    53
2011    53
2007    52
2003    51
2009    50
2010    50
2013    49
2002    49
2015    47
1997    46
2012    46
2008    45
2014    44
2001    42
2000    42
1998    40
2004    39
1996    37
Name: count, dtype: int64
```

In [312…
```
df["auto_year"].isnull().sum()
```

Out[312…
```
0
```

In [313…
```
# Observe : in this column there is no null value
```

In [314…
```
# Observe : this column has  1995 model of value  contains more  or less in 1996
```

In [315…
```
df["auto_year"].nunique()
```

Out[315…
```
21
```

In [316…
```
# Observe : this column has 21 types of value  contains
```

column 39= "fraud_reported"

In [318…
```
df["fraud_reported"].dtypes
```

Out[318…
```
dtype('O')
```

In [319…
```
# Observe : this column is object Datatype,discrete variable and correct Datatypes
```

In [320…
```
df["fraud_reported"].isnull().sum()
```

Out[320…
```
0
```

In [321…
```
# Observe : in this column there is no null value
```

In [322…
```
df["fraud_reported"].value_counts()
```

Out[322…
```
fraud_reported
N    753
Y    247
Name: count, dtype: int64
```

In [323…
```
# Observe : this column has  N  value  contains more  or less in Y
```

In [324…
```
df["fraud_reported"].nunique()
```

Out[324…
```
2
```

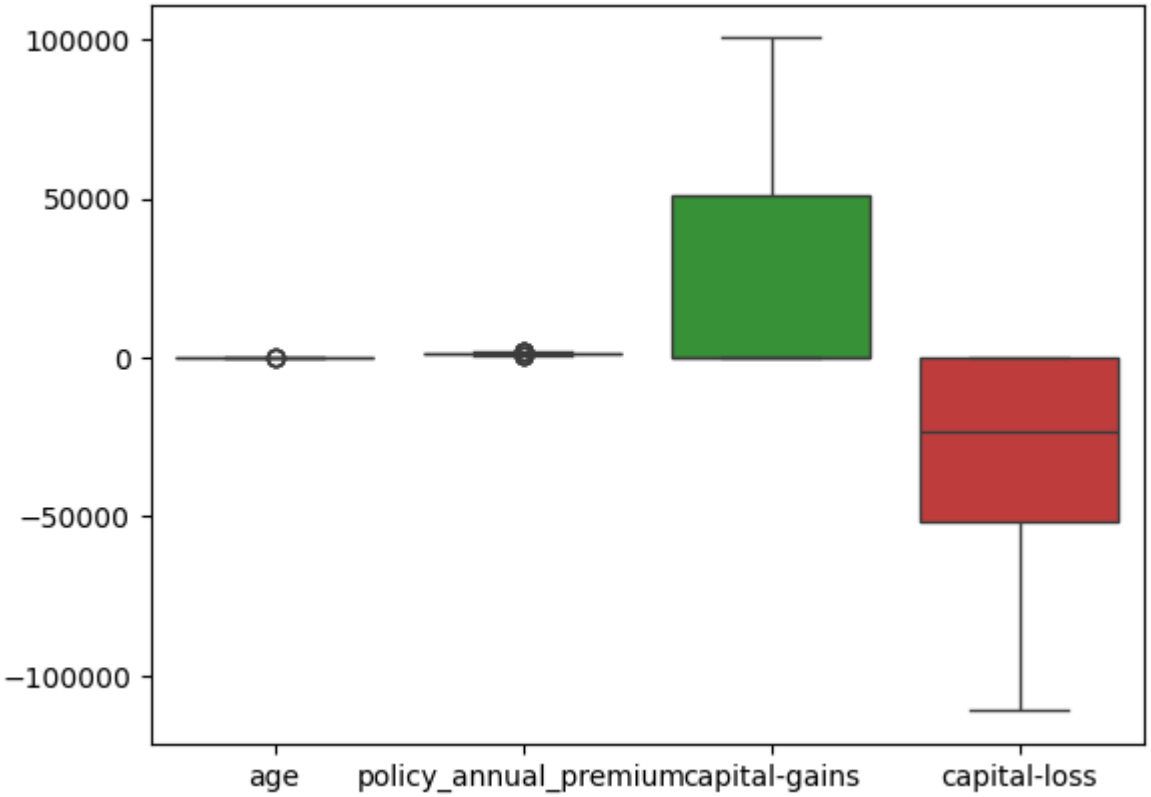
In [325…
```
# Observe : this column has 2 types of value  contains
```

In [326…
```
# seperate the continuous ,timesries,discrete variable
continuous=["age","policy_annual_premium","capital-gains","capital-loss","total_claim_amount","injury_claim",
            "property_claim","vehicle_claim"]
timeseries=["incident_date"]
discrete=["months_as_customer","policy_state","policy_csl","policy_deductable","umbrella_limit","insured_sex",
        "insured_education_level","insured_occupation","insured_hobbies","insured_relationship","incident_type"
```

```
,"collision_type", "incident_severity","authorities_contacted","incident_state","incident_city",
"incident_hour_of_the_day","number_of_vehicles_involved","property_damage","bodily_injuries",
"bodily_injuries" ,"witnesses","police_report_available","auto_make","auto_model","auto_year",
  "fraud_reported"]
```

In [327…
```
sns.boxplot(df[["age","policy_annual_premium","capital-gains","capital-loss"]])
```

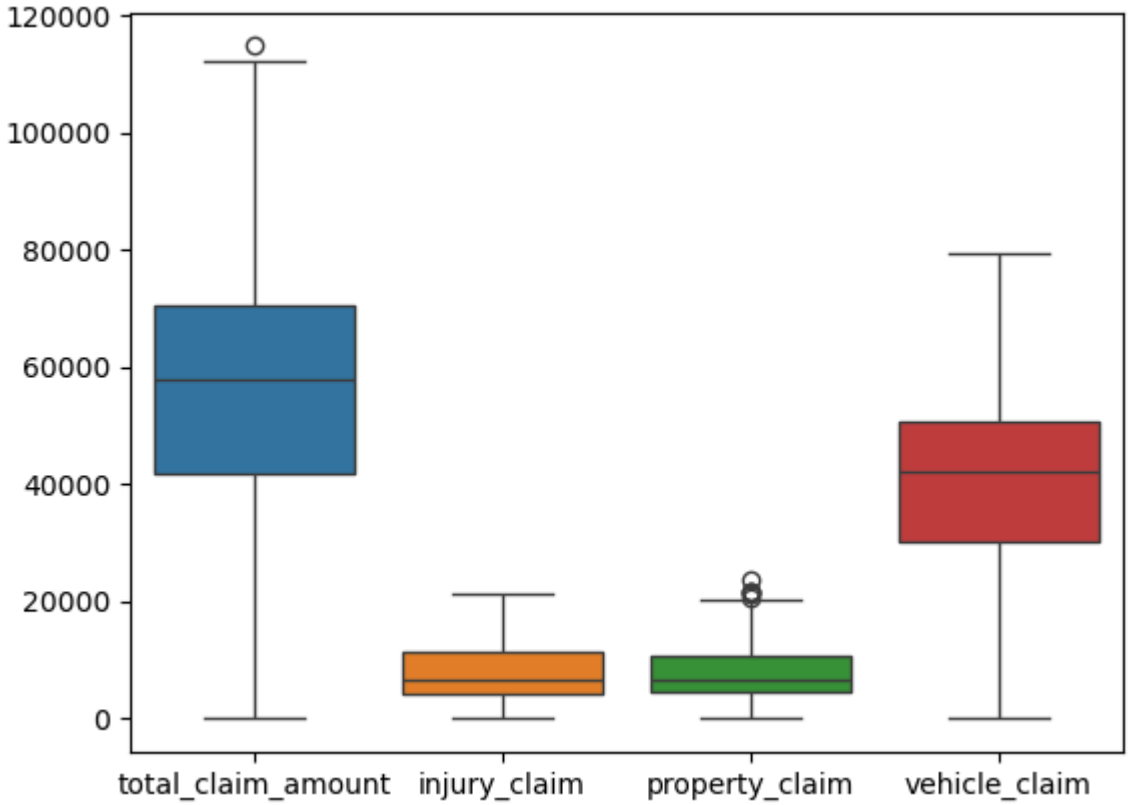
Out[327…
```
<Axes: >
```



In [328…
```
# Observe :- some outliers are available
```

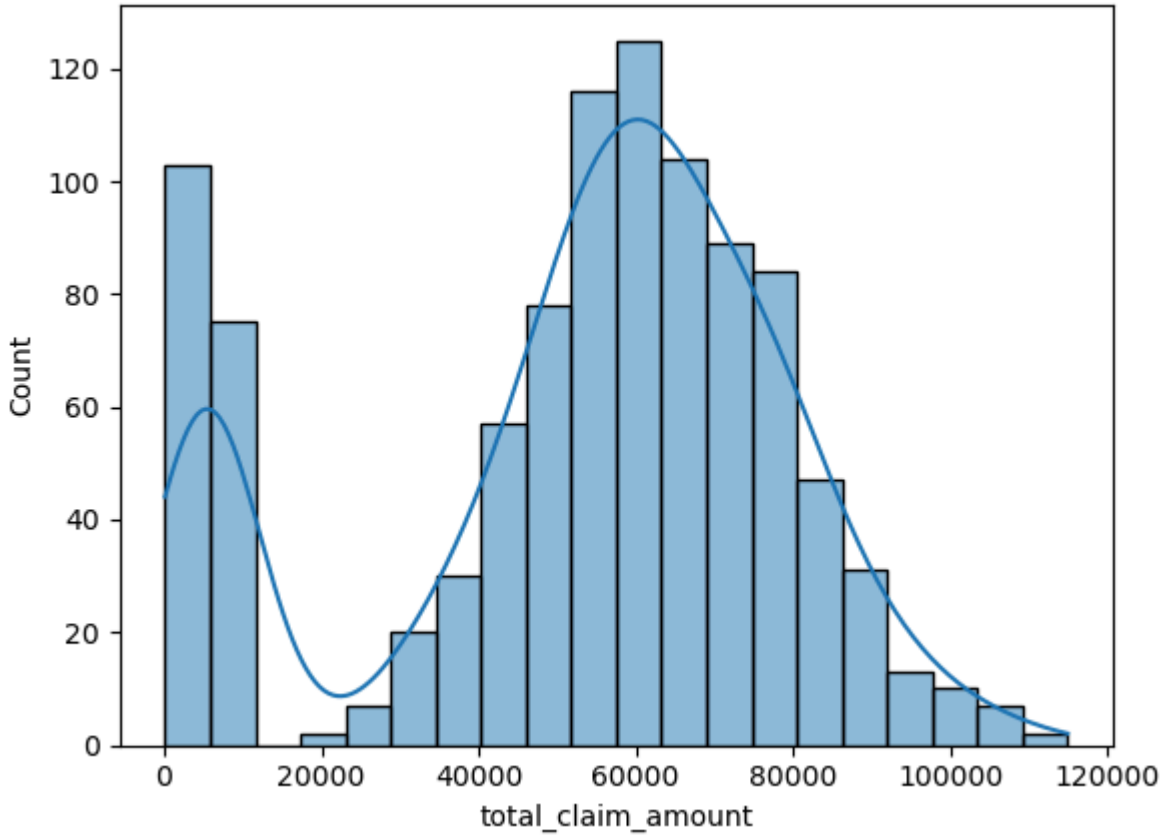
In [329…
```
sns.boxplot(df[["total_claim_amount","injury_claim","property_claim","vehicle_claim"]])
```

Out[329…
```
<Axes: >
```



In [330…
```
# Observe :- some outliers are available
```

In [331…
```
sns.histplot(df["total_claim_amount"],kde=True)
plt.show()
```

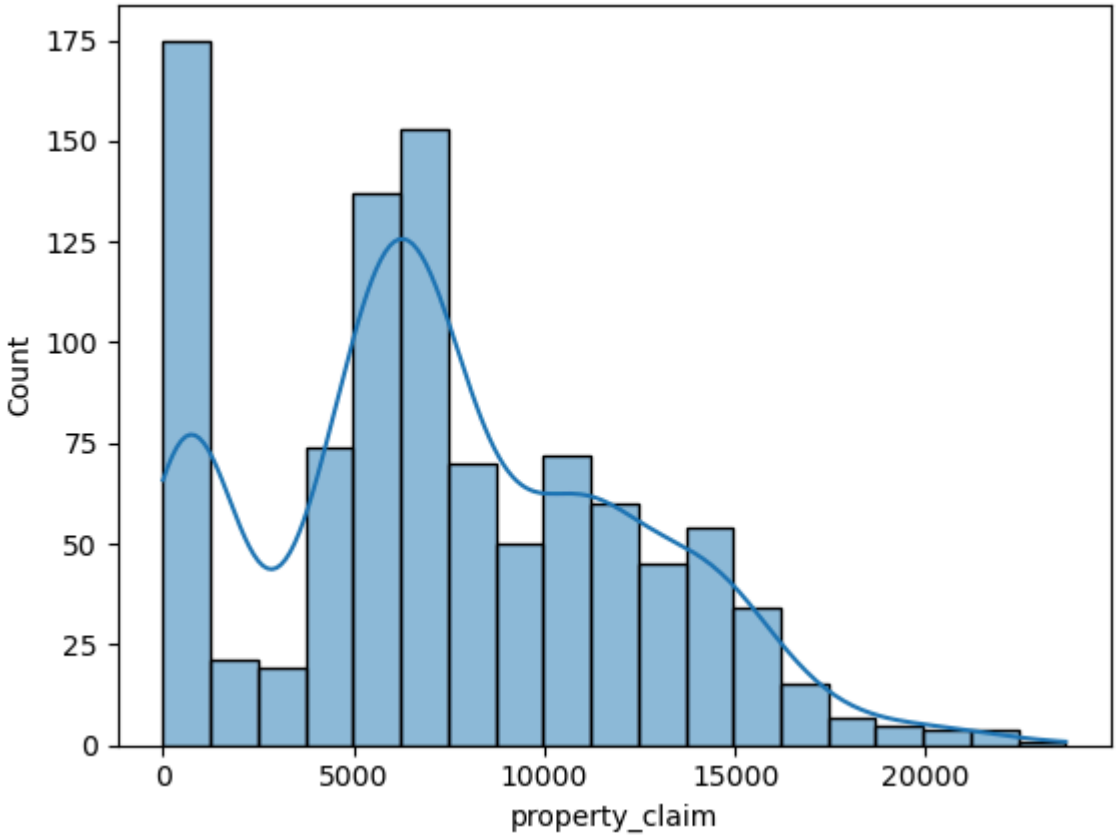


In [332…

```
## observe :- here we observe left side skewness available  in this features
```

In [333…

```
sns.histplot(df["property_claim"],kde=True)
plt.show()
```



In [334…
```
## observe :- here we observe right side skewness available  in this features
```

Data Preprocessing

In [336…
```
# Data Cleaning
```

In [337…
```
# 1>treat wrong Data types
#(umbrella_limit column has wrong data,collision_type,property_damage,police_report_available)
```

In [338…
```
df.loc[df["umbrella_limit"]==-1000000,"umbrella_limit"]=0
```

In [339…
```
# Reason :- Umbrella_limit provides additional liability coverage,
#so the limit should always be zero or positive — never negative.
```

In [340…
```
df['collision_type'] = df['collision_type'].replace('?', 'Unknown')
```

In [341…
```
# Reason:- Replace ? with a meaningful value like "Unknown"
```

In [342…
```
df["property_damage"]=df["property_damage"].replace("?","Unkown")
```

In [343…
```
# Reason:- Replace ? with a meaningful value like "Unknown"
```

In [344…
```
df["police_report_available"]=df["police_report_available"].replace("?","Unknown")
```

In [345…
```
# Reason:- Replace ? with a meaningful value like "Unknown"
```

In [346…
```
# 2>Treat missing value    ("authorities_contacted")
```

In [347…
```
df["authorities_contacted"].fillna("Uknown",inplace=True)
df["authorities_contacted"].isnull().sum()
```

Out[347…
```
0
```

In [348…
```
# it’s safe, interpretable, and doesn’t guess. that,s why i can not fill with mode
# fill with unknown
```

In [349…
```
# 3>Dimension reduction
```

In [350…
```
df.drop(columns=["policy_bind_date","insured_zip","incident_location"],inplace=True)
```

In [351…
```
# Reason : - almost each rows have unique value  that,s  simply drop them
```

In [352…
```
# treat outliers
```

In [353…
```
#Retain outliers when they might carry meaningful, real-world insights — especially in fraud detection
```

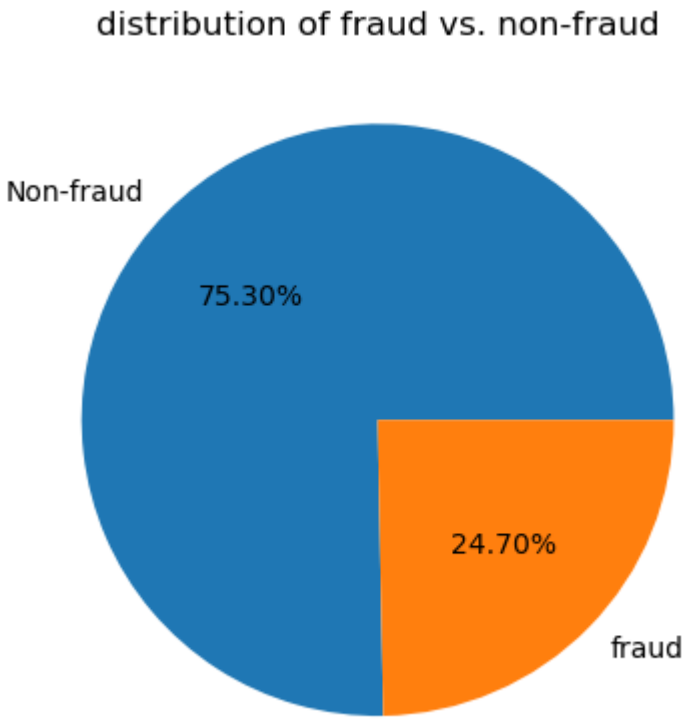
Data Analysis and Visualization

In [355…
```
# Q1. What is the distribution of fraud vs. non-fraud cases?
```

In [356…
```
df["fraud_reported"].value_counts()/len(df["fraud_reported"])*100
```

Out[356…
```
fraud_reported
N    75.3
Y    24.7
Name: count, dtype: float64
```

In [357…
```
plt.pie(df["fraud_reported"].value_counts(),labels=["Non-fraud","fraud"],autopct="%1.2f%%")
plt.title("distribution of fraud vs. non-fraud")
plt.savefig("froud.jpg")
plt.show()
```



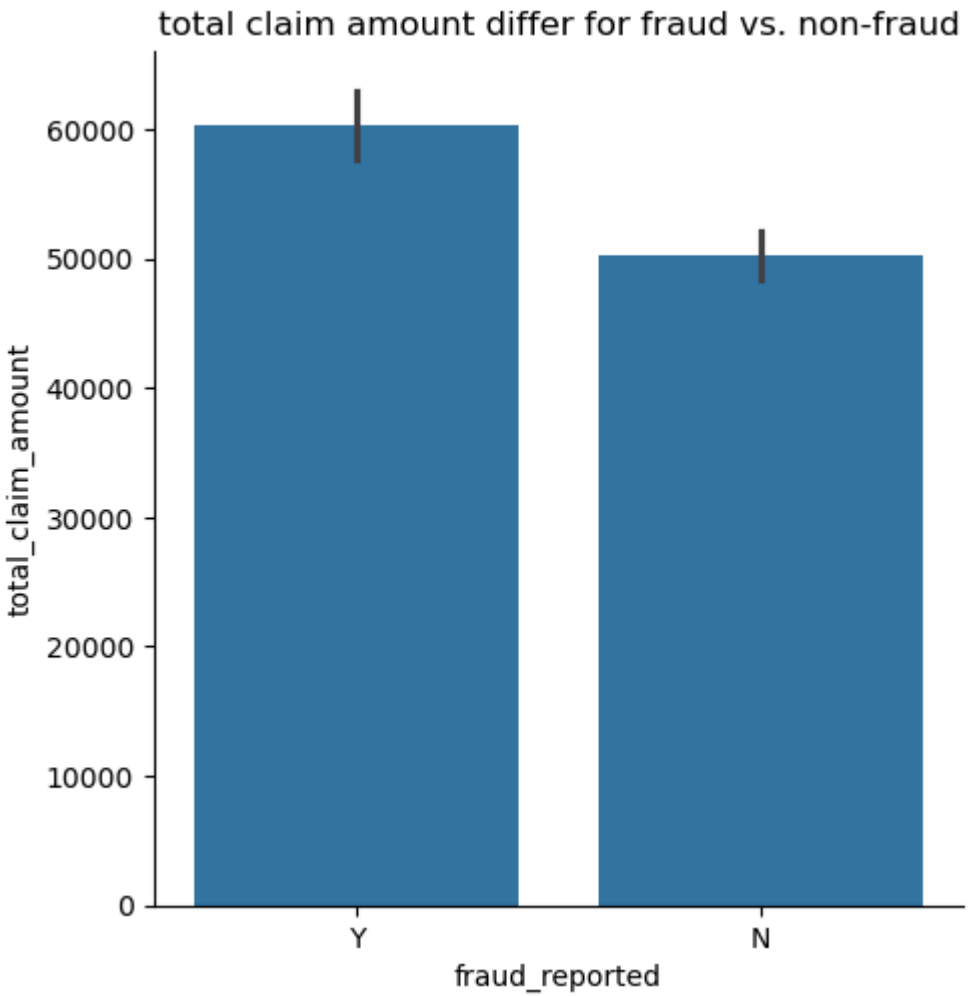
In [358…
```
# Q2. How does total claim amount differ for fraud vs. non-fraud?
```

In [359…
```
df.groupby("fraud_reported")["total_claim_amount"].mean()
```

Out[359…
```
fraud_reported
N    50288.605578
Y    60302.105263
Name: total_claim_amount, dtype: float64
```

In [360…
```
sns.catplot(x="fraud_reported",y="total_claim_amount",data=df,kind="bar")
plt.title("total claim amount differ for fraud vs. non-fraud")
```

```
plt.savefig("claim_amount.jpg")
plt.show()
```



In [361…
```
# answer: -Fraudulent claims tend to have higher total claim amounts on average.
```

In [362…
```
#Q3. Which incident types are more likely to be reported as fraud?
```

In [363…
```
corr=["witnesses",""]
```

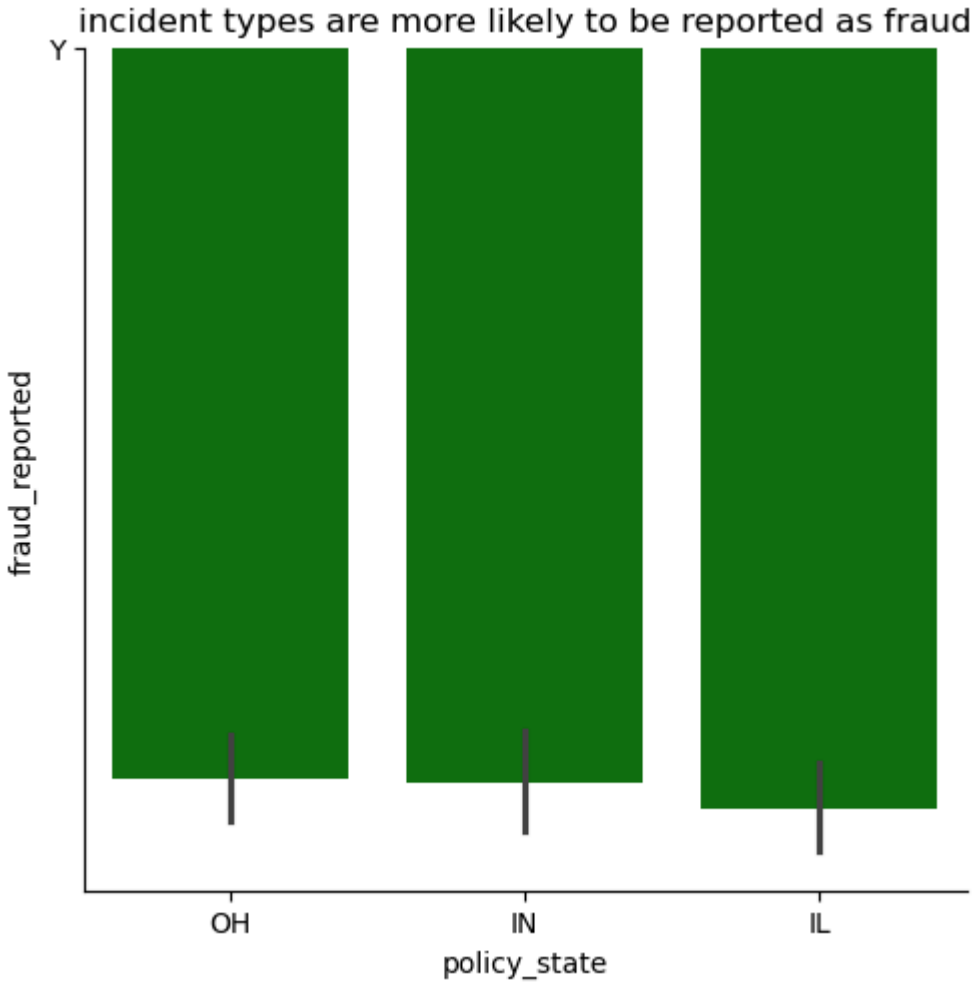
In [364…
```
pd.crosstab(df["policy_state"],df["fraud_reported"])
```

Out[364…

fraud_reported	N	Y
policy_state		
IL	261	77
IN	231	79
OH	261	91

In [365…

```
sns.catplot(y="fraud_reported",x="policy_state",data=df,kind="bar",color="green")
plt.title("incident types are more likely to be reported as fraud")
plt.savefig("froud.jpg")
plt.show()
```



In [366…
```
# answer :- IL show higher froud
```

In [367…
```
#Q4. Are older or newer cars more involved in fraud claims?
```

In [368…
```
pd.crosstab(df["auto_year"],df["fraud_reported"])
```

Out[368…

fraud_reported	N	Y
auto_year		
1995	43	13
1996	23	14
1997	34	12
1998	33	7
1999	45	10
2000	31	11
2001	33	9
2002	39	10
2003	42	9
2004	23	16
2005	42	12
2006	39	14
2007	34	18
2008	35	10
2009	39	11
2010	43	7
2011	36	17
2012	37	9
2013	34	15
2014	32	12
2015	36	11

In [369…

```
# answer:-involved in fraud claims tend to be slightly older than newer not involved.
```

In [370…

```
#Q5. Does gender influence fraud reporting?
```

In [371…]
```
pd.crosstab(df["insured_sex"],df["fraud_reported"])
```

Out[371…]

insured_sex	fraud_reported N	Y
FEMALE	411	126
MALE	342	121

In [372…]
```
# answer:- less difference between them
```

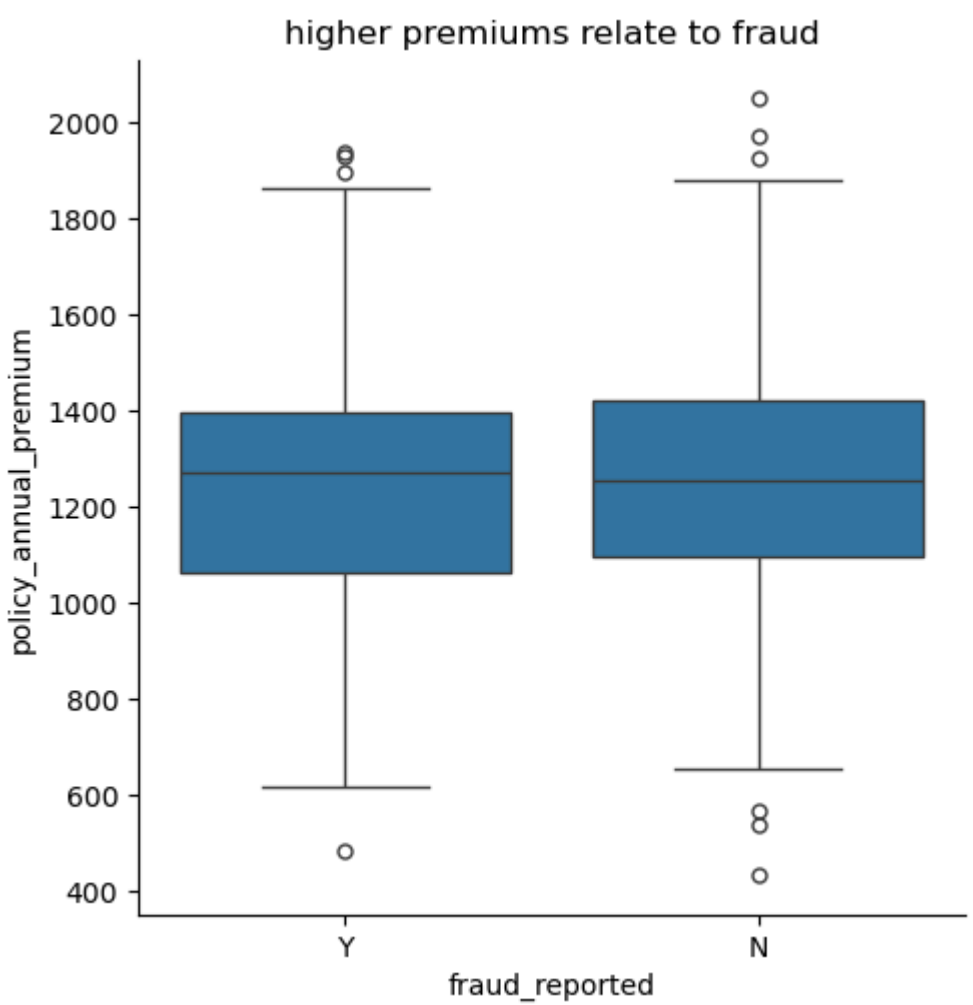
In [373…]
```
# Q6 which is the higher premiums relate to fraud?
```

In [374…]
```
df.groupby("fraud_reported")["policy_annual_premium"].max()
```

Out[374…]
```
fraud_reported
N    2047.59
Y    1935.85
Name: policy_annual_premium, dtype: float64
```

In [375…]
```
# answer= 1935.85
```

In [376…]
```
sns.catplot(x="fraud_reported",y="policy_annual_premium",data=df,kind="box")
plt.title("higher premiums relate to fraud")
plt.savefig("froud.jpg")
plt.show()
```



In [377…
```
#Q7. Which auto makes are more frequently associated with fraud?
```

In [378…
```
pd.crosstab(df["auto_make"],df["fraud_reported"])
```

Out[378…

fraud_reported	N	Y
auto_make		
Accura	55	13
Audi	48	21
BMW	52	20
Chevrolet	55	21
Dodge	60	20
Ford	50	22
Honda	41	14
Jeep	56	11
Mercedes	43	22
Nissan	64	14
Saab	62	18
Suburu	61	19
Toyota	57	13
Volkswagen	49	19

In [379…
```
# answer:-Ford,mercedes
```

In [380…
```
# Q8. Does the presence of a police report impact fraud detection?
```

In [381…
```
pd.crosstab(df["police_report_available"],df["fraud_reported"])
```

Out[381…

fraud_reported	N	Y
police_report_available		
NO	257	86
Unknown	254	89
YES	242	72

In [382…
```
# answer:- less impact on froud detection
```

In [383…
```
# Q9 How many witnesses are present in fraud cases?
```

In [384…
```
pd.crosstab(df["witnesses"],df["fraud_reported"]).sum()
```

Out[384…
```
fraud_reported
N    753
Y    247
dtype: int64
```

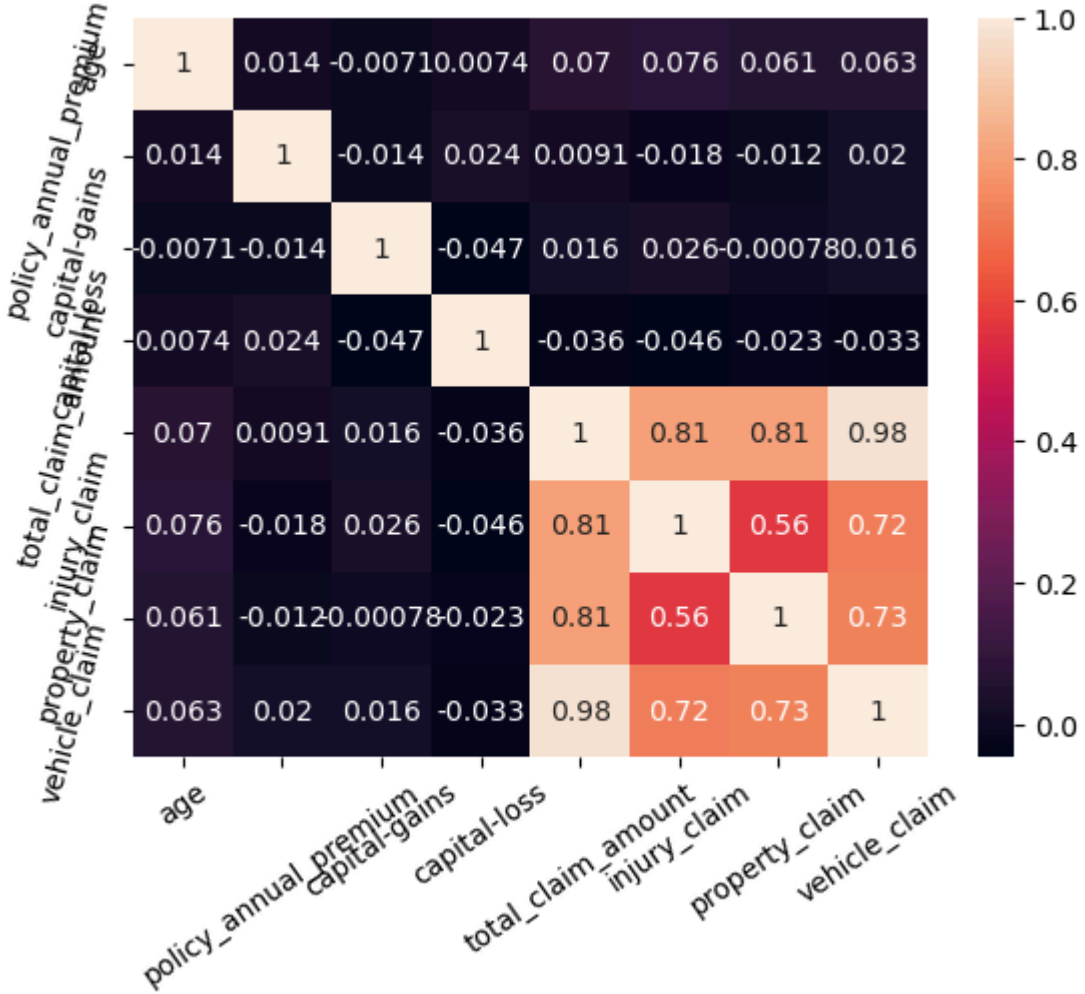
In [385…
```
# answer:- 247
```

In [386…
```
corre=df[["age","policy_annual_premium","capital-gains","capital-loss","total_claim_amount","injury_claim",
          "property_claim","vehicle_claim"]].corr()
corre
```

Out[386…

	age	policy_annual_premium	capital-gains	capital-loss	total_claim_amount	injury_claim	property_claim	vehicle_claim
age	1.000000	0.014404	-0.007075	0.007368	0.069863	0.075522	0.060898	0.062588
policy_annual_premium	0.014404	1.000000	-0.013738	0.023547	0.009094	-0.017633	-0.011654	0.020246
capital-gains	-0.007075	-0.013738	1.000000	-0.046904	0.015980	0.025934	-0.000779	0.015836
capital-loss	0.007368	0.023547	-0.046904	1.000000	-0.036060	-0.046060	-0.022863	-0.032665
total_claim_amount	0.069863	0.009094	0.015980	-0.036060	1.000000	0.805025	0.810686	0.982773
injury_claim	0.075522	-0.017633	0.025934	-0.046060	0.805025	1.000000	0.563866	0.722878
property_claim	0.060898	-0.011654	-0.000779	-0.022863	0.810686	0.563866	1.000000	0.732090
vehicle_claim	0.062588	0.020246	0.015836	-0.032665	0.982773	0.722878	0.732090	1.000000

In [387…
```
sns.heatmap(corre,annot=True)
plt.xticks(rotation=35)
plt.yticks(rotation=75)
plt.savefig("correlation of conti. variable.jpg")
plt.show()
```



In [388…

```
#Strong Positive Correlations:-
#total_claim_amount has a very strong positive correlation with vehicle_claim (0.98), property_claim (0.81),
#and injury_claim (0.81

#Weak Correlations:-
#policy_annual_premium, capital-gains, and capital-loss also generally show weak correlations with each other
# age shows very weak correlations with most other variables,

#Near-Zero or Very Weak Negative Correlations:-
#There are some very weak negative correlations, such as between age and capital-gains (-0.0071)
#or age and capital-loss (-0.0074).
```

In [389…

```
#  BY (MANISH KUMAR)
```

In [ ]:

In [ ]: